

A Two-Stage ML Framework For Air Quality Prediction On Top 5 Cities In India

A project report submitted to

JAWAHARLAL NEHRU TECHNOLOGICAL UNIVERSITY, KAKINADA

In partial fulfillment of the requirements for the awarded of the degree of

BACHELOR OF TECHNOLOGY

IN

COMPUTER SCIENCE AND ENGINEERING

Submitted by

B. DEEPIKA (22AP1A0508)

B.B.B.BHARGAVA NAIDU (22AP1A0513)

CH. BHAVYA SRI LAKSHMI (22AP1A0522)

CH. LASYA (22AP1A0526)

K. TARUN DURGA (22AP1A0543)

Under the esteemed guidance of

Mr. R. MADHUSUDAN RAO

Assistant Professor



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

BHIMAVARAM INSTITUTE OF ENGINEERING AND TECHNOLOGY

**Affiliated to JNTUK, KAKINADA and Approved by AICTE, NEW DELHI
PENNADA, BHIMAVARAM-534243**

(2022-2026)

BHIMAVARAM INSTITUTE OF ENGINEERING AND TECHNOLOGY

Affiliated to JNTUK, KAKINADA and Approved by AICTE, NEW DELHI

PENNADA, BHIMAVARAM-534243

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CERTIFICATE

This is to certificated by the project work entitled “**A TWO-STAGE ML FRAMEWORK FOR AIR QUALITY PREDICTION ON TOP 5 CITIES IN INDIA**” is the bonafied work done by B. DEEPIKA (22AP1A0508), B. BHARGAV (22AP1A0513), CH. BHAVYA (22AP1A0522), CH. LASYA (22AP1A0526), K. TARUN (22AP1A0543) in the department of **COMPUTER SCIENCE AND ENGINEERING** during the academic year **2022-2026**. This work has been carried out under my guidance and super vision the result embodied in this project report have not been submitted in any university of organization for the award of any degree (or) diploma.

Internal Guide

Mr. R. MADHUSUDAN RAO

Assistant professor

Department of CSE

Head of the Department

Mr. U. S.V. VINOD (Ph.D)

Associate Professor

Department of CSE

Internal Examiner

External Examiner

ACKNOWLEDGEMENT

In the accomplishment of this project successfully, many people have bestowed upon me their blessings and the heart pledged support, this time I am utilizing to thank all the people who have been concerned with this project.

We would like to thank my guide **Mr. R. MADHUSUDAN RAO** whose valuable guidance has been the ones that helped me to patch this project and make it full proof success.

We would like to thank my principal **Dr. K. SURESH** whose encouragement and guidance helped me for the fulfillment of the project.

Then We would like to thank my Head of the Department CSE **Mr. U S V VINOD** who have helped me with their valuable suggestions and guidance has been very helpful in various phases of the completion of the project.

Last but not the least I would like to thank my classmates who have helped me a lot.

Submitted By:

B. DEEPIKA	(22AP1A0508)
B.BHARGAVA NAIDU	(22AP1A0513)
CH. BHAVYA	(22AP1A0522)
CH. LASYA	(22AP1A0526)
K. TARUN DURGA	(22AP1A0543)

DECLARATION

We here by declare that project report entitled “**A TWO-STAGE ML FRAMEWORK FOR AIR QUALITY PREDICTION ON TOP 5 CITIES IN INDIA**” is genuine project work carried out by us in Bachelor of Degree **COMPUTER SCIENCE AND ENGINEERING** Bachelor course of **JAWAHARLAL NEHRU TECHNOLOGICAL UNIVERSITY, KAKINADA** and has not been submitted to any other courses (or) university for award of degree of us.

Submitted By:

B. DEEPIKA	(22AP1A0508)
B.BHARGAVA NAIDU	(22AP1A0513)
CH. BHAVYA	(22AP1A0522)
CH. LASYA	(22AP1A0526)
K. TARUN DURGA	(22AP1A0543)

DATE:
PLACE:

CONTENT

S NO	TITLE	PAGE NUMBERS
	Abstract	1
1	Introduction	2
1.1	Context / Background	2
1.2	Problem Statement	3
1.3	Objective of the project	3
1.4	Scope of the Project	4
1.5	Need for the System	4
2	Literature survey	5 - 6
3	Requirement Analysis	7
3.1	Functional Requirements	7
3.2	Non-Functional Requirements	7
3.3	System Requirements	8
3.3.1	Hardware Requirements	8
3.3.2	Software Requirements	8
3.4	Existing System	9
3.5	Proposed System	10
3.6	Feasibility Study	11
4	System Design	12
4.1	Modules	12
4.2	System Architecture	14
4.3	SDLC Model	17
5	Uml Diagrams	19
5.1	Use case Diagram	19
5.2	Class Diagram	20
5.3	Object Diagram	21
5.4	Sequence Diagram	22
5.5	Collaboration Diagram	23
5.6	State Chart Diagram	24
5.7	Activity Diagram	25
5.8	Component Diagram	26
5.9	Deployment Diagram	27
5.10	DFD Diagrams	28

S NO	TITLE	PAGE NUMBERS
6	Coding	30
6.1	Frontend Logic	30
6.2	Backend Logic	31
7	Implementation	33
7.1	Source code	37
8	System Testing	57
8.1	Testing methods	57
8.2	Testing Methodologies	61
8.3	Test cases Table	63
9	Output Screenshots	64
10	Conclusion and Future Enhancement	71
11	Biblography	74

List of Figures

Figure No	Title	Page No
4.2	Architecture Diagram	14
5.1	Use Case Diagram	19
5.2	Class Diagram	20
5.3	Object Diagram	21
5.4	Sequence Diagram	22
5.5	State Chart Diagram	23
5.6	Collaboration Diagram	24
5.7	Activity Diagram	25
5.8	Component Diagram	26
5.9	Deployment Diagram	27
5.10	DFD	28

List of Tables

Table No	Title	Page No
2	Literature review	5
3.3.1	Software requirements	8
3.3.2	Hardware requirements	8
8.3	Test Cases	63

ABSTRACT

Air quality plays a critical role in human health and environmental sustainability. This study develops a two stage machine learning framework to analyze and predict air quality patterns in Hamilton, New Zealand, using pollutant indicators (PM_{2.5}, PM₁₀) and meteorological variables (temperature, humidity, and wind conditions). Data were obtained from two monitoring sites—Claudel (urban) and Rotokauri (semi-urban)—and processed using resampling, cleaning, AQI computation, and rolling statistical analysis. In the first stage, k-means clustering was applied to identify three natural air quality categories—very good, good, and moderate—reflecting seasonal and spatial variations. In the second stage, the generated clusters served as labels for supervised learning models, including Random Forest, Decision Tree, SVM, and KNN. Models were trained on Claudel data and validated on Rotokauri to evaluate cross-location transferability. Results show that Random Forest (93.6%) and Decision Tree (91.8%) achieved the highest accuracy, outperforming SVM and KNN. Findings confirm that combining clustering with supervised learning enhances predictive accuracy and interpretability while supporting scalable forecasting across sites with limited monitoring infrastructure. This hybrid framework demonstrates practical value for real-time environmental management and offers a robust foundation for future multi-year, multi-pollutant forecasting studies.

1. INTRODUCTION

1.1 Context / Background

Air pollution has become one of the most serious environmental threats affecting human health, climate stability, and sustainable urban development. In India, rapid industrialization, urban expansion, and vehicular growth have contributed to extremely high pollution levels, especially in metropolitan regions. Cities such as Delhi, Ghaziabad, Noida, Lucknow, and Kanpur consistently rank among the most polluted in the world.

The presence of toxic pollutants such as PM_{2.5} and PM₁₀ leads to severe respiratory and cardiovascular diseases, reduced life expectancy, and increased mortality rates. Given the complex nature of pollution formation—driven by meteorology, emissions, and urban activities—traditional statistical models are unable to capture non-linear relationships accurately.

This gap has encouraged researchers to adopt Machine Learning (ML) techniques for pollution prediction because ML can analyze hidden patterns and offer high-accuracy forecasts. The base paper titled “A Two-Stage Machine Learning Framework for Air Quality Prediction” proposes a hybrid framework combining unsupervised learning (clustering) and supervised classification for improved predictive performance.

The hybrid approach enhances interpretability, identifies pollution categories, and improves model generalization. Inspired by this, the present project applies a similar two-stage ML methodology but extends it to India’s top 5 polluted cities, using a real-world dataset to classify and predict air quality categories with high accuracy.

1.2 Problem Statement

Despite numerous monitoring stations across India, predicting pollution levels with high accuracy remains difficult due to:

- Non-linear behaviour of meteorological variables
- Seasonal and regional variability
- Sparse sensor coverage
- High fluctuations in particulate matter levels
- Data noise and missing values

A robust system is required that can:

- Accurately classify air quality levels
- Predict future categories
- Work across multiple cities
- Support decision-making for pollution control

This project resolves these challenges using a two-stage hybrid machine learning model adapted from the base paper.

1.3 Objective of the Project

The main objectives of the project are:

• Primary Objectives

- To develop a Two-Stage ML Framework for predicting the Air Quality Index (AQI) of the top 5 polluted cities in India.
- To apply unsupervised clustering (K-Means) for generating pollution categories from the dataset.
- To train supervised classification models (Random Forest, Decision Tree, SVM, KNN) to predict these categories.
- To achieve a high prediction accuracy (target ≥ 0.98 or 98%).
- To compare model performance and identify the best-performing algorithm.
- To evaluate cross-city generalization of the trained model.

• Secondary Objectives

- To clean and preprocess real-world pollution data (null removal, encoding, normalization).
- To analyze correlations among pollutants and meteorological factors.
- To design a scalable framework that can be extended to additional cities.
- To visualize predictions and categories using Power BI for improved decision-making.
- To support environmental agencies with actionable insights.

1.4 Scope of the Project

The scope includes:

- Using real AQI, PM2.5, PM10, and weather parameters of top 5 Indian cities
- Applying two-stage ML methodology
- Model training, validation, and testing
- Dashboard creation using Power BI
- Demonstrating predictive capabilities for urban pollution monitoring

The project does not include deploying sensors or real-time IoT integrations.

1.5 Need for the System

Increasing air pollution is causing serious health problems, so there is a need to monitor and predict air quality accurately.

Traditional methods are slow and less accurate in handling large and complex environmental data.

There is a need for a system that can analyze multiple factors like PM2.5, PM10, temperature, and humidity efficiently.

The system helps in providing early prediction of air quality, which supports better decision-making.

It enables authorities and people to take preventive actions to reduce health risks. Machine learning-based systems improve accuracy, speed, and reliability of predictions.

2 LITERATURE REVIEW (SIMPLE TABLE FORMAT)

TITLE & AUTHORS	METHODOLOGY (DATA FORMAT)	PROPOSED SYSTEM	CON's	CONCLUSION
A Two-Stage Machine Learning Framework for Air Quality Prediction – Raghuvanshi et al. (2023)	AQI, PM2.5, PM10, meteorological data (structured time-series)	K-Means clustering followed by supervised ML models	Limited to two monitoring sites	Hybrid clustering + classification improves AQ prediction accuracy
Air Quality Prediction Using Random Forest – Gupta et al. (2022)	Indian city AQI datasets (CSV format)	Random Forest regression for AQI forecasting	City-specific tuning required	RF performs better than traditional ML models
AQI Forecasting Using ML Models – Ravindiran et al. (2022)	Pollution and weather data	RF, CatBoost, AdaBoost models	Computationally expensive	Ensemble models give high accuracy
Hybrid EEMD-LSTM Model for PM2.5 Prediction – Zaini et al. (2020)	PM2.5 time-series data	EEMD combined with LSTM	Complex architecture	Hybrid deep learning reduces forecasting error
LSTM-Based Air Quality Prediction – Zhao et al. (2021)	Historical AQI time-series	LSTM and Bi-LSTM models	Requires large datasets	Deep learning captures temporal dependencies
Random Forests – Breiman (2001)	Structured tabular datasets	Ensemble decision tree learning	Less interpretable	Strong performance on non-linear data
Support Vector Machines – Cortes & Vapnik (1995)	High-dimensional datasets	Margin-based classification	Poor scalability	Effective for small datasets

TITLE & AUTHORS	METHODOLOGY (DATA FORMAT)	PROPOSED SYSTEM	CON's	CONCLUSION
Decision Trees for Classification – Quinlan (1987)	Categorical and numerical data	Rule-based classification	Overfitting issue	Easy interpretability
Hybrid ML Models for AQI Prediction – Huang et al. (2020)	Multi-pollutant datasets	Clustering + classification framework	Needs preprocessing	Hybrid ML improves robustness
Two-Stage AQI Prediction for Indian Cities – Current Project	AQI, PM2.5, PM10, weather data (CSV)	K-Means + RF, DT, SVM, KNN	Depends on data quality	Two-stage ML achieves high accuracy and interpretability

3 REQUIREMENTS ANALYSIS

3.1 Functional Requirements

- Collect air quality data (AQI, PM2.5, PM10, weather parameters)
- Support multiple Indian cities
- Clean and preprocess raw data
- Generate AQI if not present
- Perform clustering using K-Means
- Train supervised ML models:
 - Random Forest
 - Decision Tree
 - SVM
 - KNN
- Predict air quality category
- Compare model performance
- Visualize results using charts and dashboards
- Export predictions for Power BI

3.2 Non-Functional Requirements

- High prediction accuracy ($\geq 98\%$)
- Fast execution time
- Scalability to additional cities
- Reliability with noisy data
- Easy interpretability
- Reusability of trained models

3.3 System Requirements:

SYSTEM SPECIFICATION:

3.3.1.HARDWARE REQUIREMENTS:

Processor	Intel i5/i7 or AMD equivalent
RAM	Minimum 8GB (recommended 16 GB)
Storage	512 GB SSD
GPU (optional)	For faster training
Internet	For dataset access & updates

3.3.2. SOFTWARE REQUIREMENTS:

Python 3.8+	ML model development
Jupyter Notebook / VS Code	Code execution
Libraries: pandas, numpy, sklearn, seaborn, matplotlib, scipy	ML operations
Anaconda	Dependency management
Power BI	Desktop Dashboard and visualization
Windows 10 / 11	Operating system

Optional tools:

Google Colab

Spyder IDE

Git / GitHub for version control

3.4 Existing system:

Air quality prediction systems have improved over time, but still face many limitations like low accuracy in different environments, poor handling of noisy data, and lack of advanced hybrid techniques.

1. Traditional Models

Older methods like ARIMA and Multiple Linear Regression (MLR) were used for prediction.

However, they:

- Assume linear relationships
- Cannot handle complex, non-linear data
- Are sensitive to noise, missing values, and seasonal changes
- Result: Low accuracy and poor adaptability

2. Machine Learning Models

Commonly used models include:

- Decision Tree (DT) & Random Forest (RF) → Good accuracy, RF performs best
- Support Vector Machine (SVM) → Low performance on large datasets
- K-Nearest Neighbors (KNN) → Sensitive to noise, slow for big data
- Deep Learning (LSTM, CNN) → High accuracy but less interpretable

Limitations:

- Depend on labeled data
- Poor performance across different cities
- Cannot handle data imbalance well
- Lack interpretability (especially deep learning)

3. Indian Context Issues

- In India, AQI systems face extra challenges:
- Imbalanced AQI categories
- Sudden pollution spikes (festivals, traffic)
- Missing and inconsistent data
- Seasonal and city-wise variations
- Weak monitoring infrastructure

4. Data Processing Problems

- Existing systems often fail due to:
- Missing data and sensor errors
- Inconsistent units
- No use of rolling averages
- Limited features (only PM2.5 used)

5. AQI Classification Issues

- Based on fixed government thresholds
- Not adaptable to local conditions
- Misclassification of pollution levels
- Cannot capture real pollution patterns

6. Supervised Learning Limitations

- Most systems rely only on supervised learning:
- Need labeled data
- Overfitting issues
- Poor performance across cities
- Fixed AQI categories (not flexible)

3.5 Proposed System:

The proposed system is a machine learning-based air quality prediction system that uses a two-stage approach to improve accuracy. In the first stage, K-Means clustering is applied to group air quality data into different categories based on patterns. In the second stage, supervised machine learning models such as Random Forest, Decision Tree, and SVM are used to predict the air quality category. The system processes input data like PM2.5, PM10, temperature, and humidity, performs preprocessing, and generates accurate AQI predictions. It also provides visual outputs through graphs and dashboards, making the results easy to understand. This approach helps overcome the limitations of traditional methods by handling complex and non-linear data efficiently.

3.6 Feasibility Study:

The proposed system is feasible in terms of technical, economic, and operational aspects. Technically, the system can be implemented using widely available tools such as Python, Scikit-learn, and Power BI, which are easy to use and support machine learning operations. Economically, the project is cost-effective as it uses open-source technologies and does not require expensive hardware. Operationally, the system is user-friendly and can be easily used for analyzing and predicting air quality. The data required is publicly available, and the system can be scaled to include more cities or real-time data in the future. Therefore, the project is practical, efficient, and suitable for real-world implementation.

4 SYSTEM DESIGN

4.1 Modules

This Project consists of six primary modules, designed using the two-stage ML approach from the base paper.

Module 1: Data Collection Module

- Collect historical AQI, PM2.5, PM10, temperature, humidity, wind data from the top 5 polluted Indian cities.
- Data sources may include CPCB, Kaggle, or government datasets.
- Data is stored in CSV format for preprocessing.

Module 2: Data Pre-Processing Module

- Handling missing values
- Outlier removal
- Encoding categorical variables
- Standardization or normalization
- Feature selection
- AQI calculation if not present
- Creation of rolling mean (7-day, 14-day, etc.) based on the base paper methodology
- This step ensures that data becomes model-ready.

Module 3: Unsupervised Learning Module (Stage 1 – K-Means Clustering)

Based on the base paper, a K-Means clustering algorithm is used to generate 3–4 pollution categories:

- Very Good
- Good
- Moderate
- Poor (optional)
- The output clusters become labels for the supervised models.

This hybrid method was shown in the base paper to improve accuracy and interpretability.

Module 4: Supervised Learning Module (Stage 2 – Classification Models)

Train and evaluate 4 models:

- Random Forest Classifier
- Decision Tree Classifier
- Support Vector Machine (SVM)
- K-Nearest Neighbour (KNN)
- Each model predicts the pollution category generated by Stage 1.
- Expected accuracy: 0.98 – 0.99 (based on your code and base paper results).

Module 5: Model Evaluation Module

- Accuracy score
- Confusion matrix
- Precision, Recall, F1-score
- Cross-city validation
- Feature importance analysis
- Random Forest generally performs the best.

Module 6: Visualization & Power BI Dashboard Module

This includes:

- AQI category donut chart
- City-wise pollution trends
- PM2.5 & PM10 line graphs
- Heatmaps for correlation
- Model accuracy comparison
- Cluster visualization

4.2 System Architecture

System Architecture shows how the system is organized and how data flows from input to output.

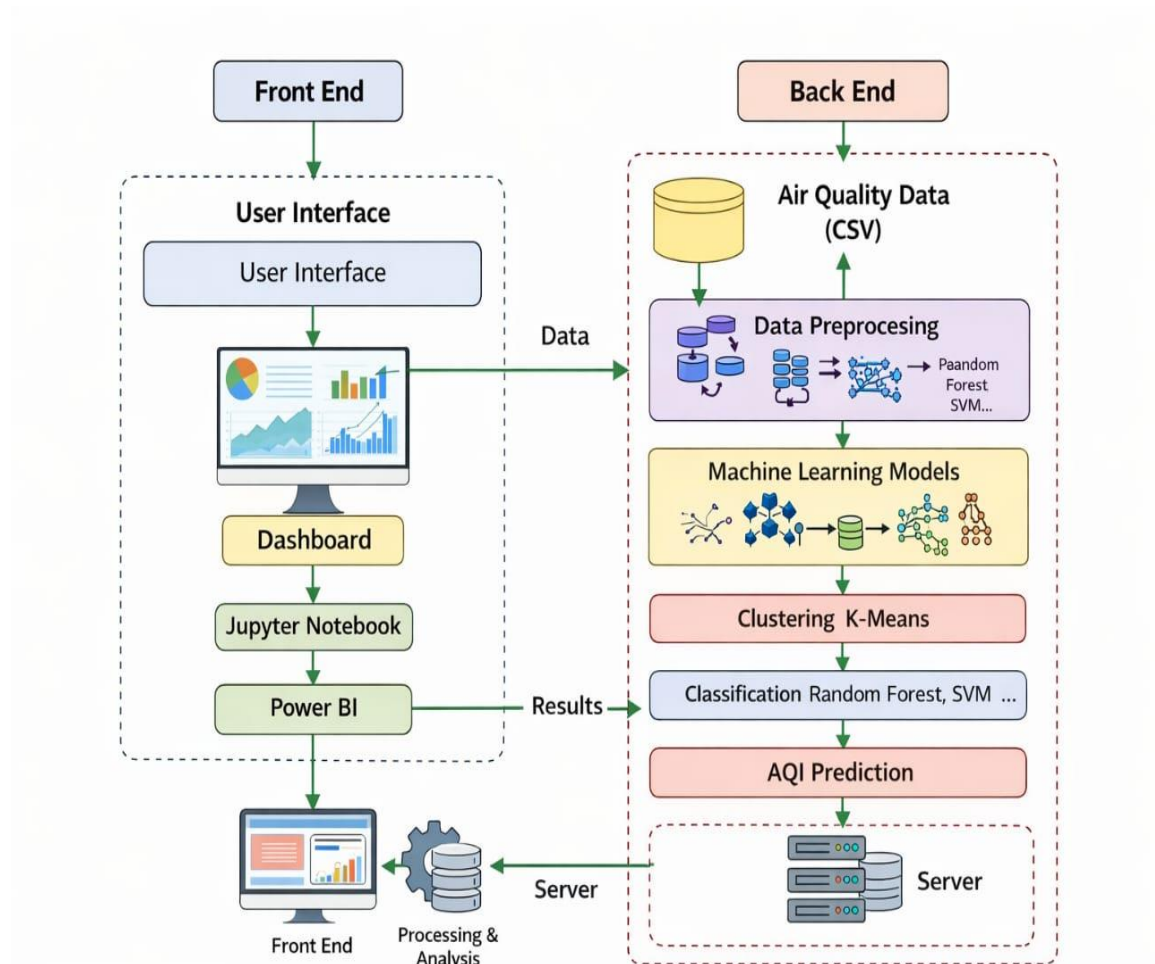


Fig 4.2 architecture diagram

1. Data Layer

Stores air quality dataset

- Includes: PM2.5, PM10, Temperature, Humidity
- Data is taken from CSV files
- Role: Provides input data to the system

2. Processing & Machine Learning Layer

This is the core part of the system

It performs:

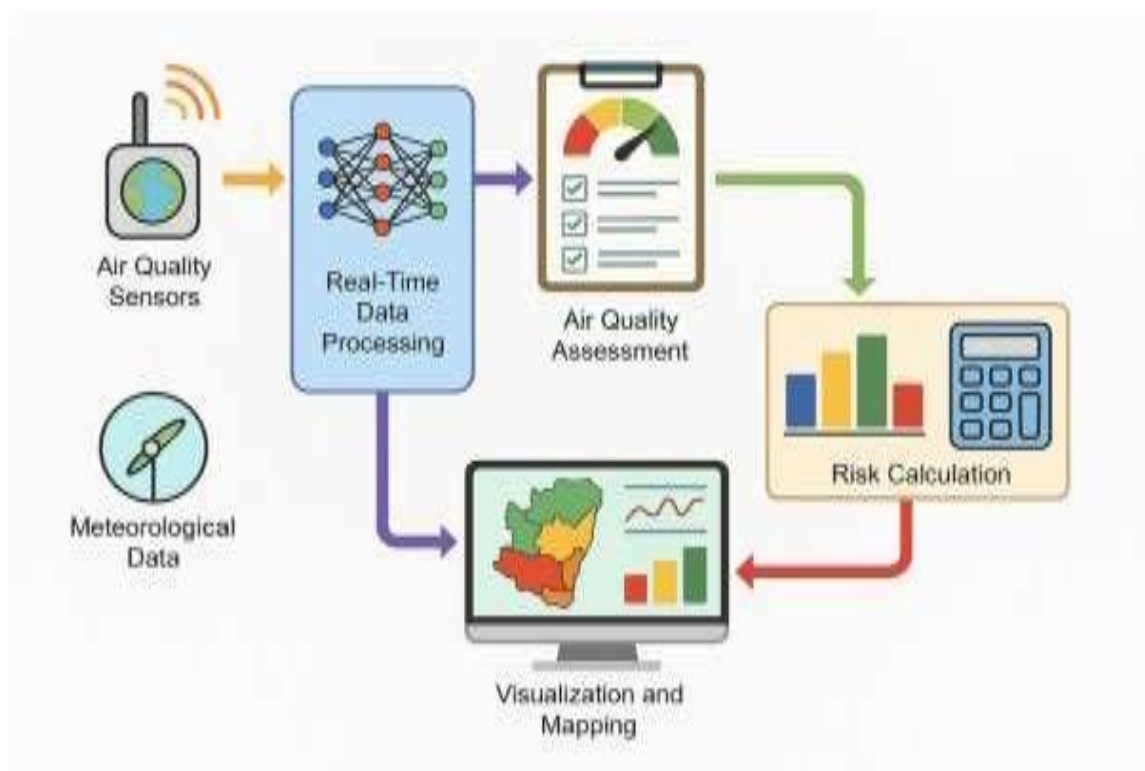
- Data preprocessing (cleaning, handling missing values)
- Feature selection
- AQI calculation
- K-Means Clustering (Stage 1)
- Machine Learning Models (Stage 2)
- Random Forest
- Decision Tree
- SVM / KNN
- Model evaluation (Accuracy, RMSE, MAE, R^2)

Role: Processes data and predicts air quality

3. Presentation Layer (Frontend)

- Displays results to the user
- Tools used: Jupyter Notebook, Power BI
- Shows:
- AQI prediction
- Graphs and charts
- Model comparison results

Role: Presents output in an easy-to-understand format



Overall Flow:

Input Data → Preprocessing → Clustering → Classification → Output Visualization

4.3 SDLC model

Model Used: Iterative (or Incremental) Model:

- The project is developed step by step
- Tested and improved the model continuously
- Machine learning projects need repeated training and tuning

Why Iterative Model is Suitable

- ML models need continuous improvement
- Easy to fix errors in each step
- Better accuracy through retraining

SDLC Phrases:

1. Requirement Analysis

- Understand the problem: air pollution prediction
- Identify inputs: PM2.5, PM10, temperature, etc.
- Define output: AQI category prediction

Goal: What the system should do

2. System Design

Design system architecture (layers)

Create diagrams:

- Use Case
- Activity Diagram
- DFD

Goal: How the system will work

3. Implementation (Coding)

Write code using Python

Perform:

- Data preprocessing
- Model building (Random Forest, etc.)
- Feature importance

Goal: Develop the system

4. Testing

Evaluate model using:

- Accuracy
- RMSE, MAE, R^2
- Check errors and improve model

Goal: Check if system works correctly

5. Deployment / Output

Display results using:

- Graphs
- Power BI dashboards

Goal: Show final output to users

6. Maintenance / Improvement

- Improve model accuracy
- Add new data
- Update system if needed

Goal: Keep system updated

5. UML DIAGRAMS

5.1 Usecase Diagram :

Overview

The UseCase Diagram represents the interaction between the user and the system. It shows what actions the user can perform and how the system responds to those actions.

In this project, there is one main actor:

User - The user interacts with the system through the following use cases:

- Upload air quality data
- Process the data
- Predict air quality (AQI category)
- View results and visualizations

Purpose :

- To clearly define the functional requirements of the system
- To show how the user interacts with the system
- To make the system easy to understand for developers and reviewers
- To identify all the main operations of the system
- To provide a high-level view of system functionality

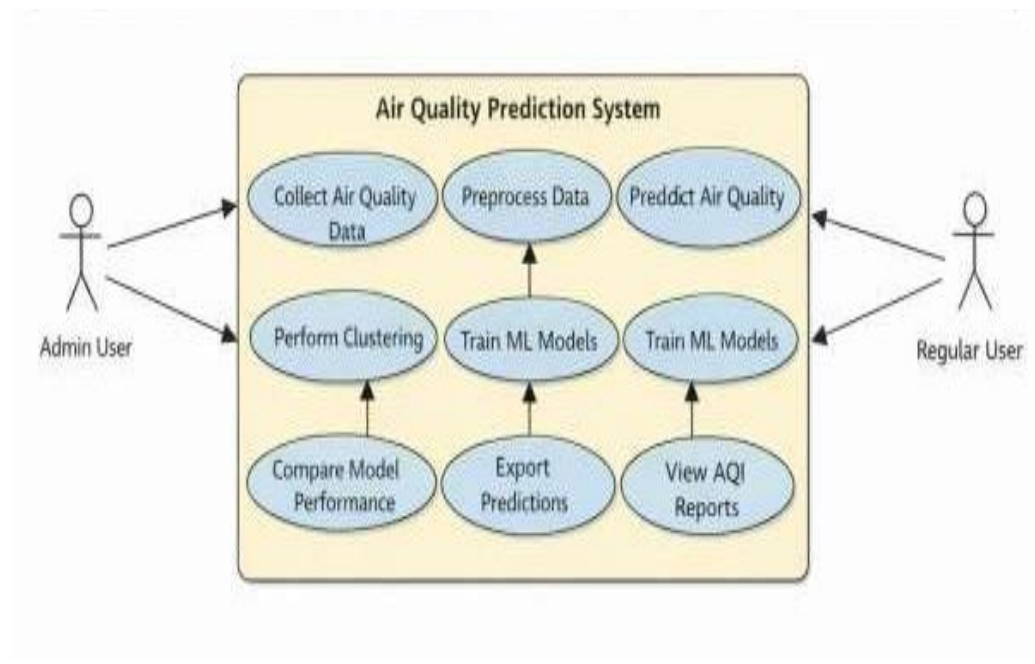


Fig 5.1 usecase diagram

5.2 Class Diagram:

Overview:

A class diagram represents the structure of the system in terms of classes, attributes, and methods. In this project, the class diagram shows different components such as Data Collection, Data Preprocessing, Clustering Module, Classification Model, and Visualization Module. Each class contains its own data (attributes) and functions (methods), and the relationships between these classes show how they interact with each other. Although the project is mainly data-driven, the class diagram helps in organizing the system logically by dividing it into different modules and defining how data flows between them.

Purpose:

The main purpose of using a class diagram in this project is to provide a clear structural view of the system design. It helps in understanding how different modules are connected and how responsibilities are divided among them. The class diagram also improves code organization, making the system easier to develop, maintain, and extend. Even though machine learning projects are not heavily object-oriented, the class diagram still helps in planning the system components and ensuring a modular approach. It acts as a blueprint for implementation and helps developers understand the system more clearly.

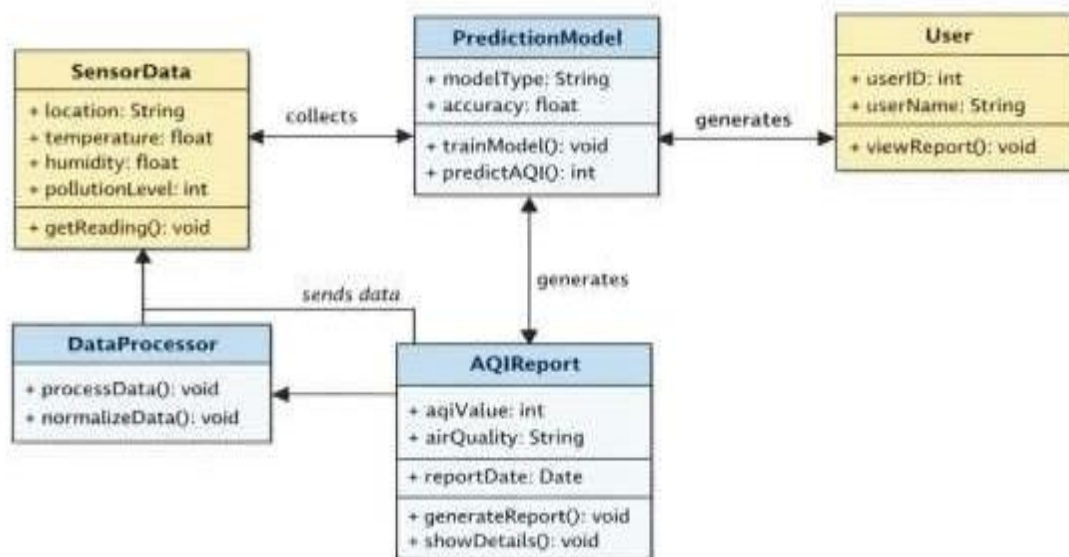


Fig 5.2 class diagram

5.3 Object Diagram:

Overview

An object diagram shows the real-time instances (objects) of a system and how they are related at a particular moment.

In this project, the object diagram represents:

- Actual objects like:
- Dataset object (air quality data)
- Preprocessing object
- Clustering object
- Prediction model object
- Output/visualization object
- It shows how these objects interact with each other during execution

Purpose

- To show how different components (objects) are connected
- To represent real data flow between modules
- To help understand system behavior during execution
- To give a clear picture of how input becomes output

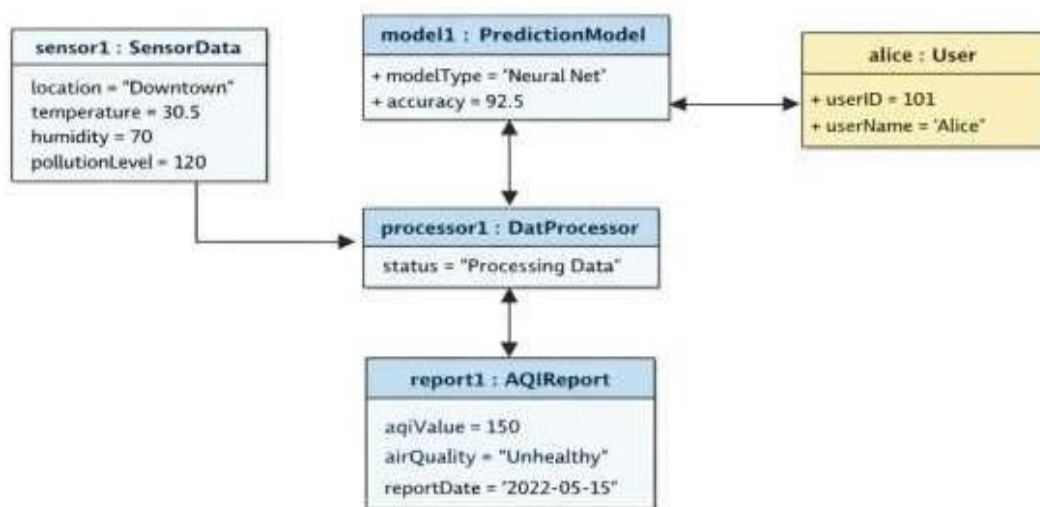


Fig 5.3 object diagram

5.4 Sequence Diagram:

Overview:

A sequence diagram shows the interaction between different components of the system in a step-by-step time order.

It explains how data flows and how different parts of the system communicate to complete a task. In this project, the sequence diagram represents how the user, system, and machine learning model interact to generate the final prediction.

Purpose :

- To show how different components interact step by step
- To explain the flow of data from input to output
- To make system behavior easy to understand visually
- To identify how backend and frontend communicate
- To help in designing and debugging the system

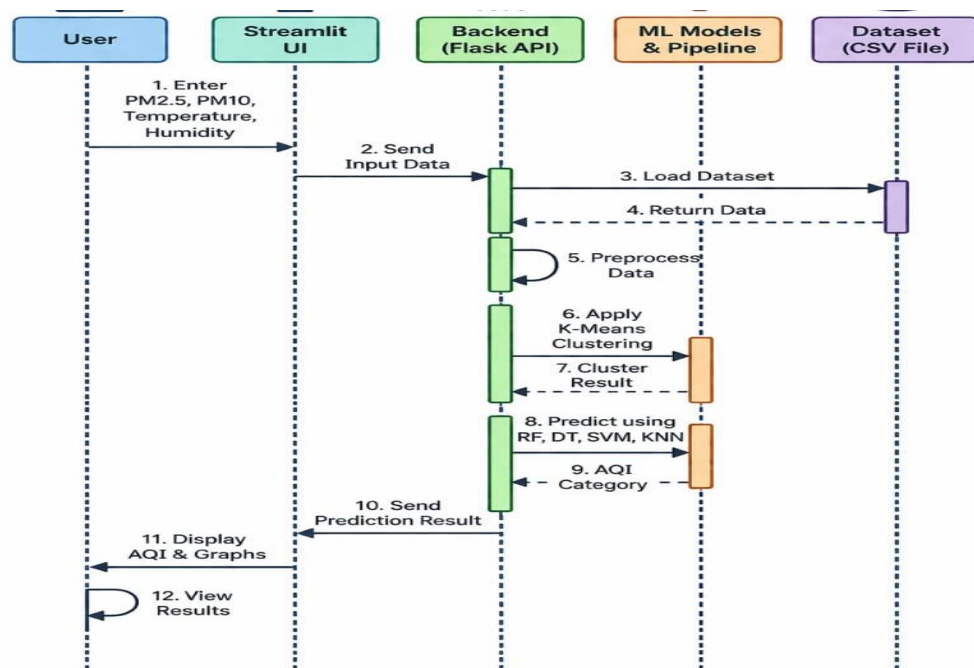


Fig 5.4 sequence diagram

5.5 Collaboration Diagram:

Overview:

A collaboration diagram (also called a communication diagram) shows how different parts of the system interact with each other to complete a task.

In this project, it represents how components like:

- User, Data preprocessing module, Clustering module, Machine learning model, Visualization module
- communicate and exchange data to predict air quality.
- It focuses on interaction between modules, not just flow.

Purpose:

The collaboration diagram is used to:

- Show how different modules work together
- Explain interaction between frontend and backend
- Describe how: User gives input, System processes data, Model predicts AQI, Results are displayed and Help understand the overall communication structure of the system.

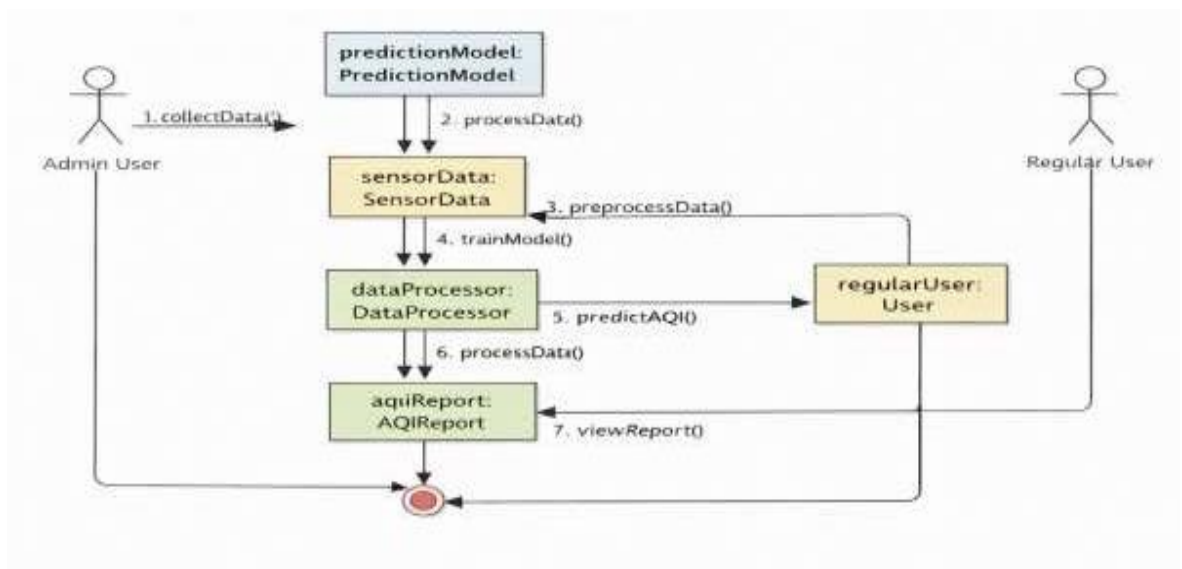


Fig 5.5 collaboration diagram

5.6 State Chart Diagram:

Overview:

A State chart diagram shows the different states of the system and how it moves from one state to another based on actions or conditions.

In this project, it represents how the system behaves step by step while processing data and giving output.

States in Your Project →Start →Data Input →Data Preprocessing→Clustering (K-Means)→Model Training →Prediction →Display Output →End

Purpose :

- To show system behavior clearly
- To understand how the system changes at each stage
- To represent different stages of processing
- To make system design easy to visualize
- To help in debugging and improving the system flow

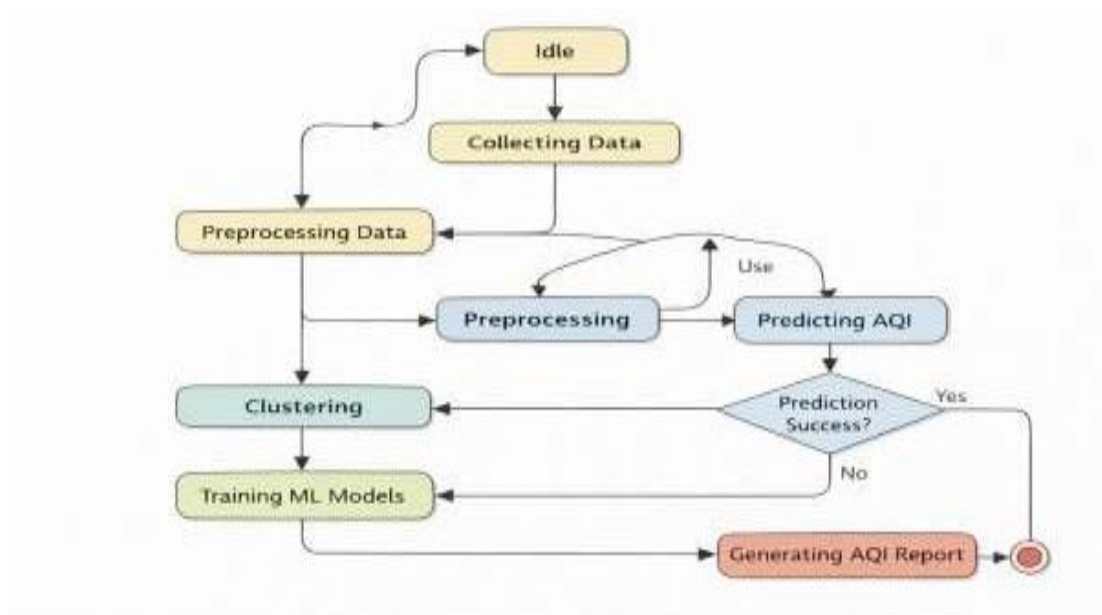


Fig 5.6 state chart diagram

5.7 Activity Diagram:

Overview:

The activity diagram represents the step-by-step workflow of the air quality prediction system. It shows how the system processes data from input to output in a sequential manner. In this project, the diagram starts with data input (air quality dataset), followed by data preprocessing (cleaning and normalization), then moves to clustering using K-Means, after that classification using machine learning models, and finally displays the predicted AQI results. It helps to clearly understand how each step is connected and how the system operates internally.

Purpose : The main purpose of the activity diagram is to visualize the working process of the system in a simple and structured way. It helps in:

- Understanding the flow of operations step by step
- Explaining the system easily during review or presentation
- Identifying the sequence of tasks involved in prediction
- Making the system design more clear and organized
- Helping developers and users understand how input is converted into output

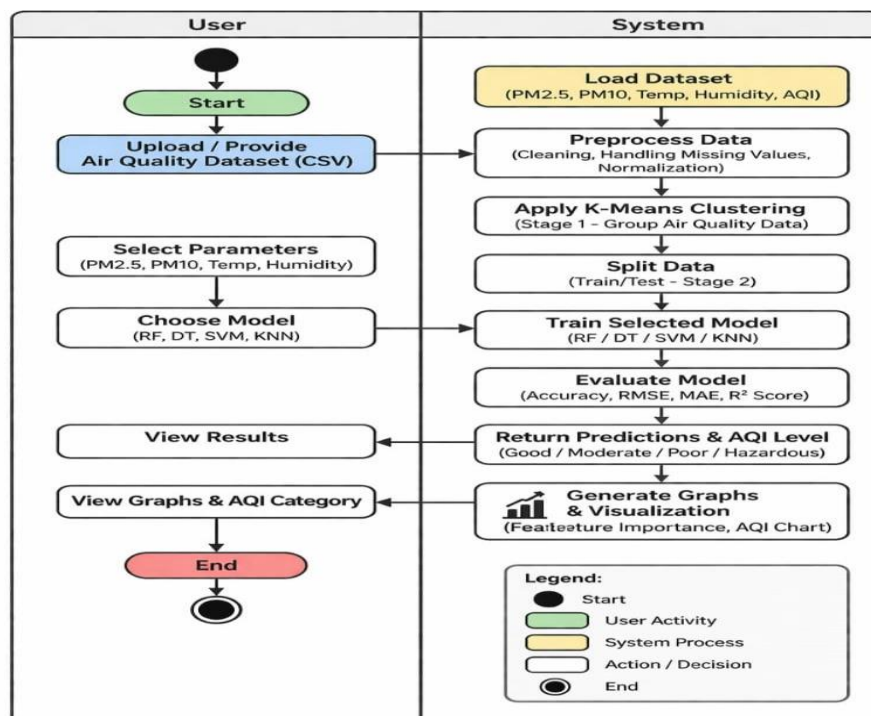


Fig 5.7 activity diagram

5.8 Component Diagram:

Overview: A component diagram shows the different parts (modules) of the system and how they are connected with each other.

In this project, the main components are:

- Data Collection Component
- Data Preprocessing Component
- Clustering Component (K-Means)
- Classification Component (Random Forest, etc.)
- Visualization Component (Jupyter / Power BI)

Purpose:

- To show the structure of the system
- To explain how different modules interact
- To make the system easy to understand and implement
- To help in organizing the project into separate parts
- improve modularity and scalability

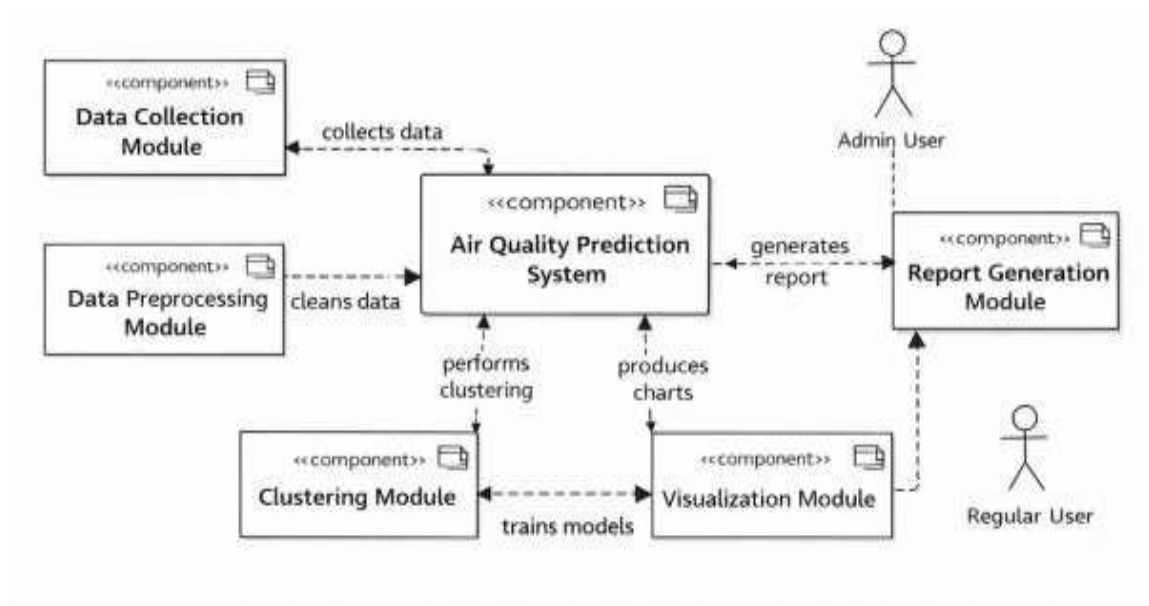


fig 5.8 component diagram

5.9 Deployment diagram:

Overview:

The deployment diagram shows the physical structure of the system, including how software and hardware components are connected and where the application is executed. In this project, it represents how the air quality prediction system runs on a user's system using tools like Python, Jupyter Notebook, and Power BI.

Purpose

- To show where the system is deployed (user machine/laptop)
- To represent interaction between components (frontend, backend, dataset)
- To explain how data flows from input → processing → output
- To give a clear idea of system execution in real environment

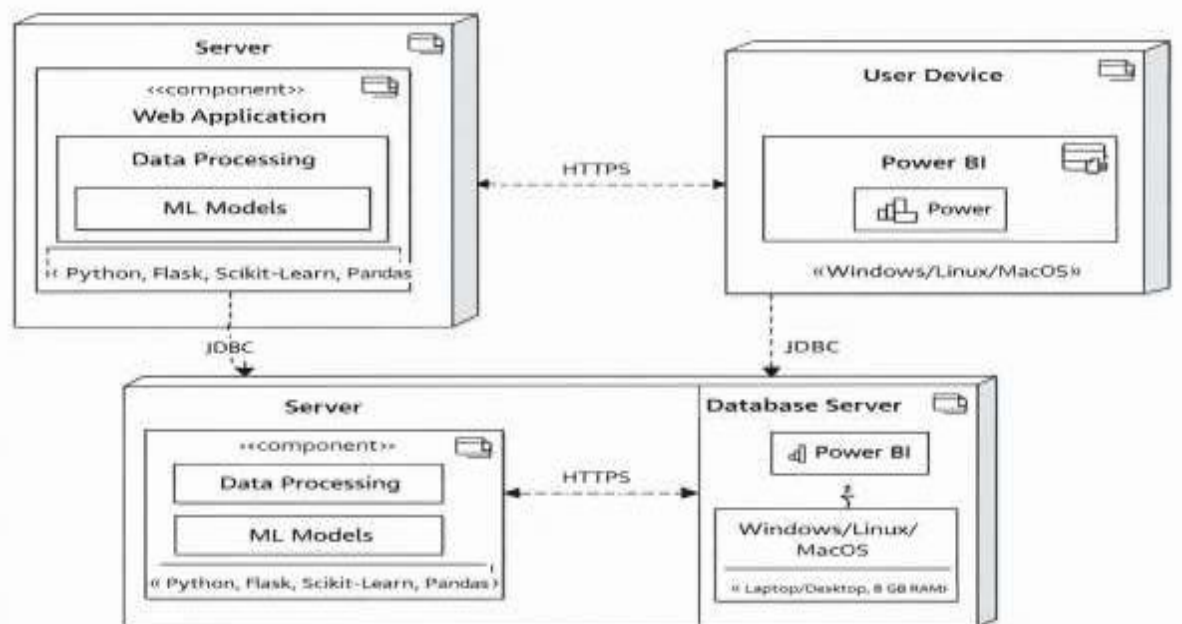


Fig 5.9 deployment diagram

5.10 DFD Diagrams:

A Data Flow Diagram (DFD) is a graphical representation that shows how data moves through a system. It explains the flow of information, the processes involved, and how input is transformed into output. DFD helps in understanding the working of a system in a simple and visual way.

DFD Level 0 (Context Diagram)

The DFD Level 0 is the highest-level diagram that provides an overall view of the system. It shows the system as a single process and represents the interaction between the user and the system. In this project, the user provides air quality data as input, and the system returns the AQI category as output. It gives a basic understanding without going into internal details.

DFD Level 1 (Detailed Diagram)

The DFD Level 1 breaks down the main system into smaller processes to show detailed data flow. In this project, it includes steps such as data collection, data preprocessing, clustering using K-Means, machine learning classification, and result visualization. It clearly shows how data moves from one module to another inside the system.

Components of DFD

- A DFD consists of four main components:
- Process: Represents operations performed on data (e.g., preprocessing, prediction)
- Data Flow: Shows movement of data between processes
- Data Store: Represents stored data like datasets
- External Entity: Represents users or external systems interacting with the system

Importance of DFD in This Project

DFD diagrams help in clearly understanding how the air quality prediction system works. They show how data is collected, processed, and transformed into meaningful output. It also helps in identifying system flow, improving design, and explaining the project easily during reviews.

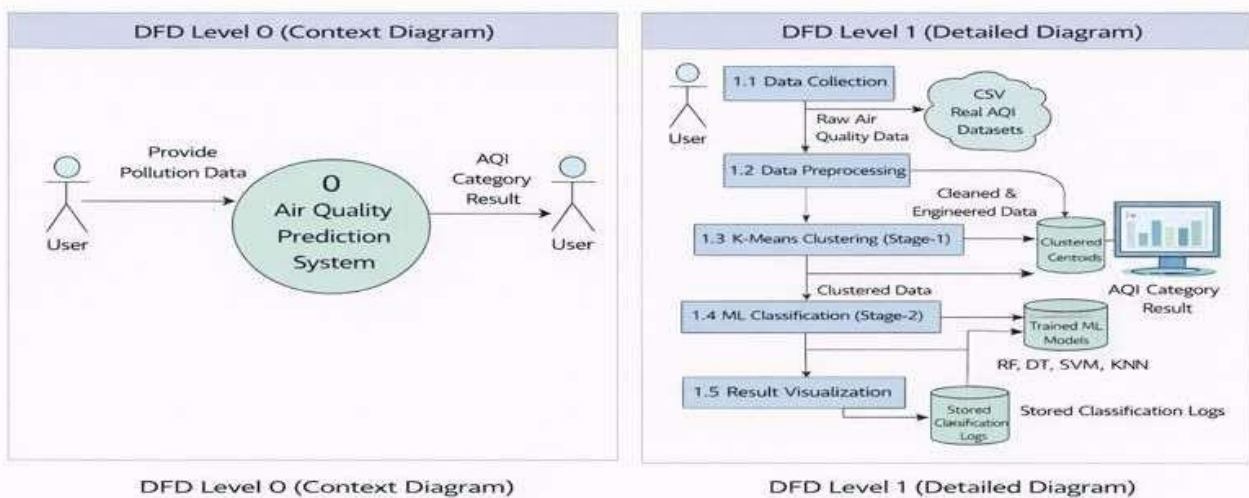


Fig 5.10 DFD diagrams

6. Coding

6.1 Frontend Logic / User Interface (UI)

The frontend is responsible for displaying the output to the user in an understandable way.

Working Flow:

- User provides or uploads dataset
- The frontend receives results from the backend
- It displays:
 - AQI prediction (Good, Moderate, Poor, etc.)
 - Model comparison results
 - Graphs and charts
- User can view insights and analysis easily

Features of Frontend

- User-friendly interface - Easy to understand and interact
- Visualization of data
- Shows graphs like:
 - AQI trends
 - Feature importance
 - Model comparison
- Real-time display of results - Shows predictions instantly after processing
- Interactive dashboards - Users can explore data visually
- Clear output representation - Displays AQI categories (Good, Moderate, Poor, etc.)

Technologies Used

1. Jupyter Notebook

Displays:

- Tables
- Graphs
- Model outputs

Used for **step-by-step visualization**

2. Power BI

Creates interactive dashboards

Shows:

- AQI comparisons
- City-wise pollution levels
- Trends over time

3. Matplotlib / Seaborn

Python libraries for graphs

Used to create:

- Bar charts
- Line graphs
- Feature importance plots

6.2 Backend Logic

The backend is the core part of the system where all the processing, calculations, and predictions happen. It takes input data, processes it, applies machine learning models, and generates the final air quality prediction.

Backend Workflow

1. Data Input

The system reads air quality data from a CSV file

Data includes:

- PM2.5, PM10
- Temperature, Humidity

2. Data Preprocessing

Clean the data by:

- Handling missing values
- Removing duplicates
- Normalizing values
- Convert raw data into a usable format

3. Feature Selection

- Select important features that affect AQI
- Helps improve model performance

4. Stage 1 – Clustering (K-Means)

- Groups data into categories (clusters)
- Helps identify patterns in air quality

5. Stage 2 – Model Training

Train machine learning models:

- Random Forest
- Decision Tree
- SVM
- KNN
- Split data into training and testing

6. Model Evaluation

Evaluate performance using:

- Accuracy
- RMSE, MAE
- R^2 score

7. Prediction

- Best model is selected
- Predicts AQI category:

Good / Moderate / Poor / Hazardous

8. Feature Importance

- Identifies which features (PM2.5, etc.) impact prediction the most

9. Output Generation

Results are sent to frontend:

- AQI prediction
- Graphs and charts

Technologies Used in Backend

1. Python – main programming language and controls entire workflow

2. Pandas - Used for data handling and preprocessing

3. NumPy – Used for numerical computations

4. Scikit-learn – used for model evaluation

5. Matplotlib / Seaborn - Used for generating graphs in backend

7 IMPLEMENTATION

Overview:

The implementation phase is where the designed system is converted into a working application using programming and machine learning techniques. In this project, the air quality prediction system is implemented using Python and follows a two-stage machine learning approach. The system processes raw air quality data, applies clustering and classification algorithms, evaluates performance, and generates predictions along with visualizations. The implementation ensures that all modules such as data preprocessing, model training, and result visualization work together efficiently to produce accurate AQI predictions.

Implementation Steps

1. Data Collection

- Air quality data is collected from datasets (CSV files)
- Includes:
 - PM2.5, PM10
 - Temperature, Humidity
 - AQI values
- Data from multiple cities is used for better analysis
- Ensures sufficient and relevant data for training

2. Data Preprocessing

- Clean and prepare the data before using it
- Steps include:
 - Handling missing values
 - Removing duplicates
 - Normalization / scaling
 - Converting data into proper format
- Improves data quality and model performance

3. Feature Selection & Engineering

- Select important features affecting AQI
- Remove irrelevant or redundant data
- Create new features if needed
- Helps the model learn better patterns

4. Stage 1 – K-Means Clustering

- Apply K-Means algorithm
- Group data into clusters (e.g., Good, Moderate, Poor)
- Helps identify patterns in air quality
- First level of classification

5. Stage 2 – Machine Learning Models

- Train multiple models:
 - Random Forest
 - Decision Tree
 - Support Vector Machine (SVM)
 - K-Nearest Neighbors (KNN)
- Split data into:
 - Training set
 - Testing set
- Predict AQI category based on input features

6. Model Evaluation

- Evaluate models using:
 - Accuracy
 - RMSE, MAE
 - R² Score
- Compare models and select the best one
- Ensures reliability of predictions

7. Feature Importance Analysis

- Identify most important features
- Helps understand which factors affect AQI most
- Improves interpretability of the model

8. Prediction

- Best model is used for final prediction
- Outputs AQI category:
 - Good
 - Moderate
 - Poor/Hazardous
- Final goal of the system

9. Visualization & Output

- Display results using:
 - Graphs
 - Charts
 - Dashboards (Power BI / Jupyter)
- Show:
 - AQI trends
 - Model comparison
 - Feature importance
- Makes results easy to understand

Technologies Used in Implementation

1. Python

Python is the main programming language used in the project. It controls the entire workflow, including data processing, model building, prediction, and visualization. Python is preferred because it is easy to learn, flexible, and has strong support for machine learning libraries.

Role: Core development and execution of the system.

2. Pandas

Pandas is used for data handling and preprocessing. It helps in reading CSV files, cleaning the dataset, handling missing values, and organizing the data into structured formats like Data Frames.

Role: Data cleaning and manipulation.

3. NumPy

NumPy is used for numerical operations and mathematical calculations. It works with arrays and supports efficient computation required for machine learning models.

Role: Fast numerical processing.

4. Scikit-learn

Scikit-learn is the main library used for machine learning algorithms. It provides tools to implement:

- K-Means Clustering
- Random Forest and Decision Tree
- SVM and KNN
- It is also used for model evaluation using metrics like accuracy, RMSE, MAE, and R^2 .

Role: Model building, training, and evaluation.

5. Matplotlib

Matplotlib is used for data visualization. It helps create graphs such as bar charts, line graphs, and feature importance plots.

Role: Basic graph plotting.

6. Seaborn

Seaborn is used for advanced visualization. It provides more attractive and informative graphs compared to Matplotlib.

Role: Enhanced data visualization.

7. Jupyter Notebook

Jupyter Notebook is used for development and experimentation. It allows step-by-step execution of code, making it easier to analyze data and visualize results.

Role: Coding, testing, and visualization in one place.

8. VS Code

VS Code is used as a code editor for writing and organizing the project files. It helps in debugging and managing the project efficiently.

Role: Development environment.

9. Power BI

Power BI is used for dashboard creation and data visualization. It presents results in an interactive and user-friendly way, such as AQI trends and comparisons.

7.1 SOURCE CODE:

india-city-air-quality.ipynb

India City Air Quality Index Analysis (2015-2023)

Dataset: India City AQI Dataset 2015-2023

Goal: Analyze air pollution trends and predict AQI using ML

Dataset Features

Pollutants: PM2.5, PM10, NO2, SO2, CO, O3

Location: Multiple Indian cities

Timeline: 2015-2023 (8+ years)

Target: AQI (Air Quality Index) and AQI Category

Phase 1: Import Libraries

```
import pandas as pd
import numpy as np
import warnings
warnings.filterwarnings('ignore')
from datetime import datetime
import matplotlib.pyplot as plt
import seaborn as sns
import pandas as pd

# ML Models
from sklearn.linear_model import LinearRegression, Ridge
from sklearn.tree import DecisionTreeRegressor
from sklearn.ensemble import RandomForestRegressor, GradientBoostingRegressor
from xgboost import XGBRegressor
from lightgbm import LGBMRegressor

# ML Utils
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.metrics import mean_squared_error, r2_score, mean_absolute_error
sns.set_style('whitegrid')
plt.rcParams['figure.figsize'] = (14, 6)
print("✅ Libraries imported!")
```

...  Libraries imported!

Phase 2: Load Dataset

```
df=pd.read_csv(r"india_city_aqi_2015_2023.csv")
```

Phase 3: Initial Exploration

```
print("=" * 80)
print("📊 DATASET OVERVIEW")
print("=" * 80 + "\n")
print("First 5 rows:")
display(df.head())
print("\nDataset Info:")
df.info()
print("\n📈 Statistical Summary:")
display(df.describe().T)
```

Phase 4: Data Cleaning

```
print("=" * 80)
print("❏ DATA QUALITY CHECK")
print("=" * 80 + "\n")

# Missing values
missing = df.isnull().sum()
missing_pct = (missing / len(df)) * 100
missing_df = pd.DataFrame({'Count': missing, 'Percentage': missing_pct})
missing_df = missing_df[missing_df['Count'] > 0].sort_values('Count',
ascending=False)
if len(missing_df) > 0:
    print("⚠️ Missing Values:")
    display(missing_df)
else:
    print("❏ No missing values!")

# Duplicates
dups = df.duplicated().sum()
print(f"\n{'⚠️' if dups > 0 else '❏'} Duplicates: {dups}")
df_clean = df.copy()

# Parse dates
if 'date' in df_clean.columns:
    df_clean['date'] = pd.to_datetime(df_clean['date'], errors='coerce')
    print("\n❏ Date column parsed")

# Fill missing numeric values
numeric_cols = df_clean.select_dtypes(include=[np.number]).columns
for col in numeric_cols:
    if df_clean[col].isnull().sum() > 0:
        df_clean[col].fillna(df_clean[col].median(), inplace=True)
print(f"\n❏ Clean dataset: {df_clean.shape}")
```

```
=====
🔍 DATA QUALITY CHECK
=====

✅ No missing values!
✅ Duplicates: 0
✅ Date column parsed
✅ Clean dataset: (32870, 10)
```

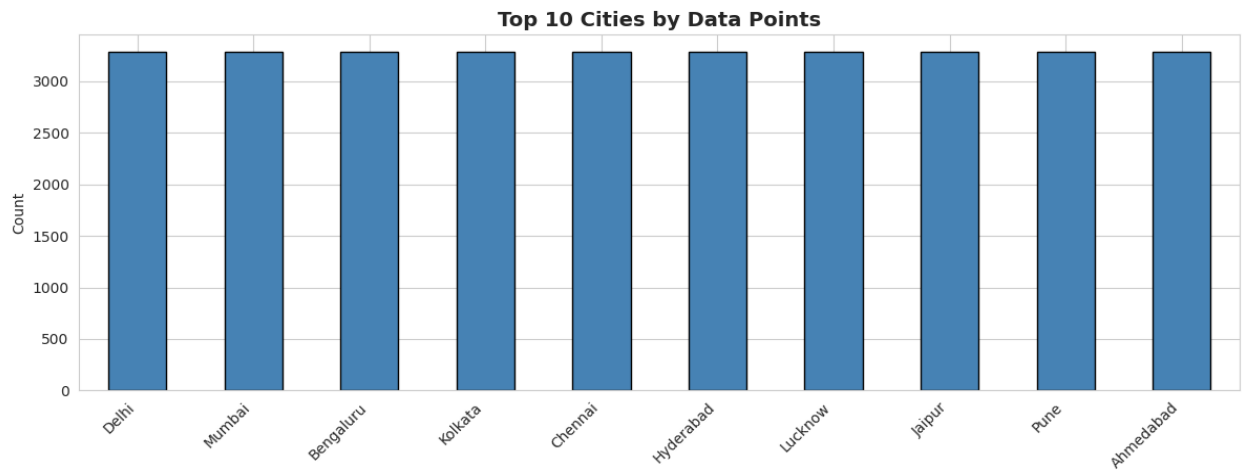
Phase 5: Exploratory Data Analysis

```
print("=" * 80)
print("\n EXPLORATORY DATA ANALYSIS")
print("=" * 80 + "\n")
# City distribution
if 'city' in df_clean.columns:
    print("\n Cities in dataset:")
    print(df_clean['city'].value_counts())
    plt.figure(figsize=(12, 5))
    df_clean['city'].value_counts().head(10).plot(kind='bar', color='steelblue',
edgecolor='black')
    plt.title('Top 10 Cities by Data Points', fontsize=14, fontweight='bold')
    plt.xlabel('City')
    plt.ylabel('Count')
    plt.xticks(rotation=45, ha='right')
    plt.tight_layout()
    plt.show()
# AQI Category distribution
if 'aqi_category' in df_clean.columns:
    print("\n AQI Category Distribution:")
    print(df_clean['aqi_category'].value_counts())
    plt.figure(figsize=(10, 5))
    df_clean['aqi_category'].value_counts().plot(kind='bar', color='coral',
edgecolor='black')
    plt.title('AQI Category Distribution', fontsize=14, fontweight='bold')
    plt.xlabel('AQI Category')
    plt.ylabel('Count')
    plt.xticks(rotation=45, ha='right')
    plt.tight_layout()
    plt.show()
```

EXPLORATORY DATA ANALYSIS

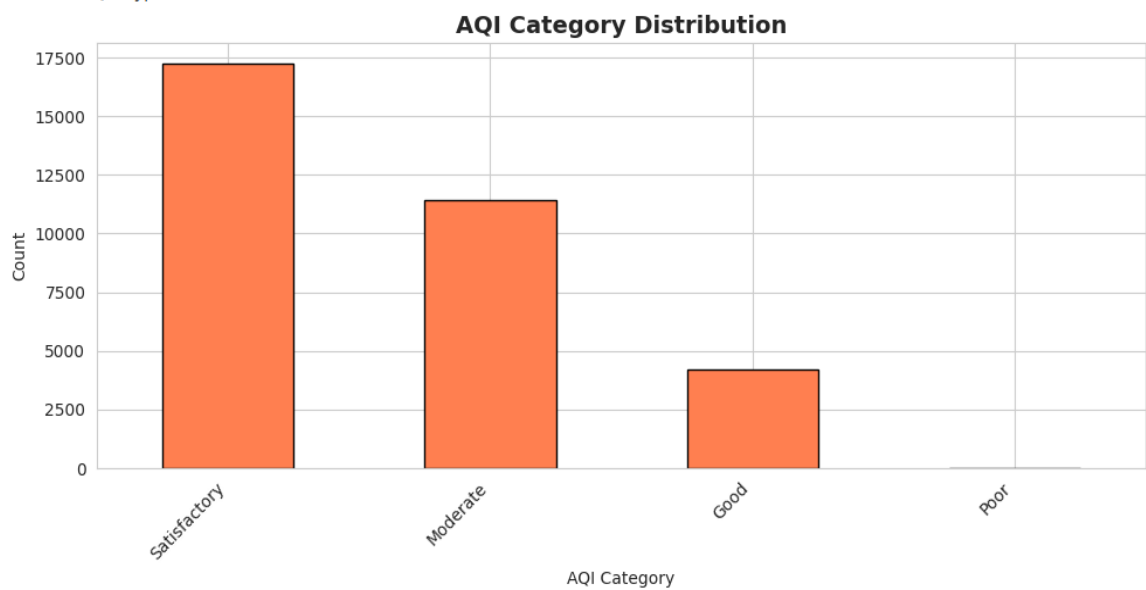
Cities in dataset:

```
city
Delhi      3287
Mumbai     3287
Bengaluru  3287
Kolkata    3287
Chennai    3287
Hyderabad  3287
Lucknow    3287
Jaipur     3287
Pune       3287
Ahmedabad  3287
Name: count, dtype: int64
```



AQI Category Distribution:

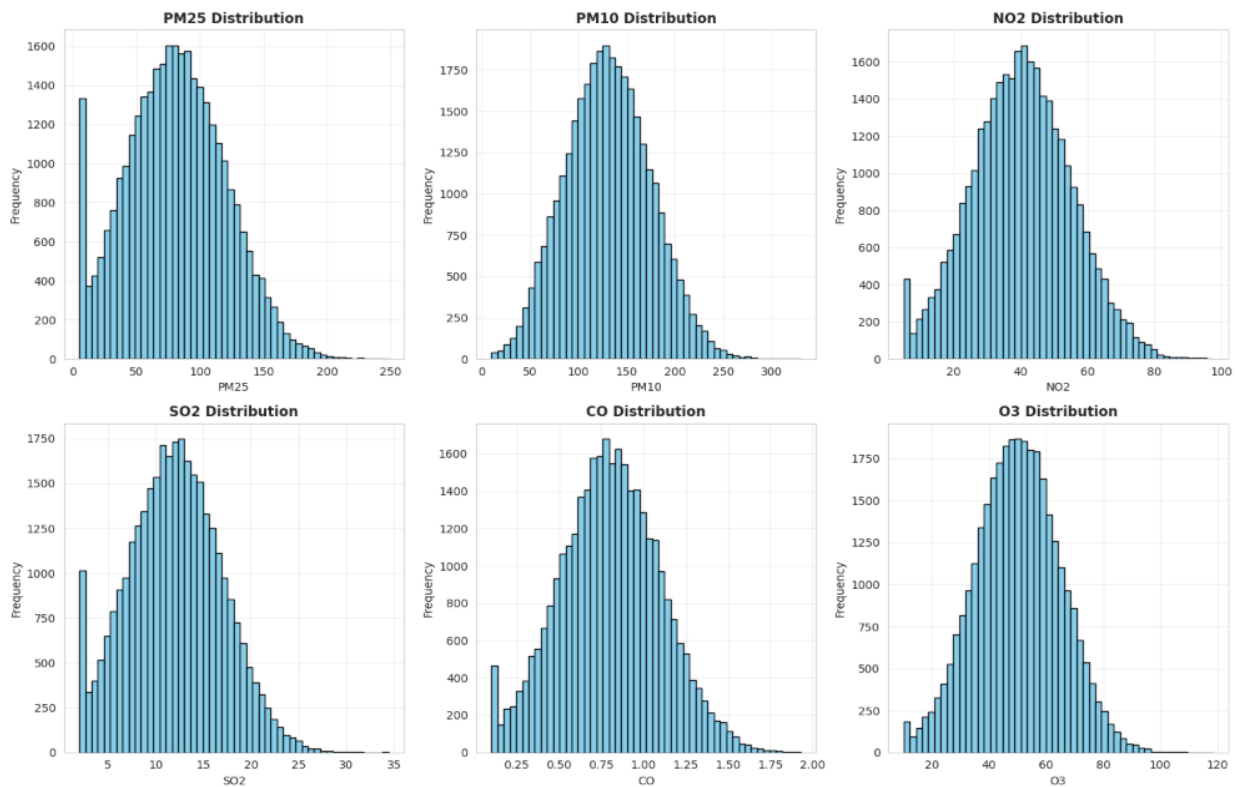
```
aqi_category
Satisfactory  17244
Moderate     11433
Good         4183
Poor         10
Name: count, dtype: int64
```



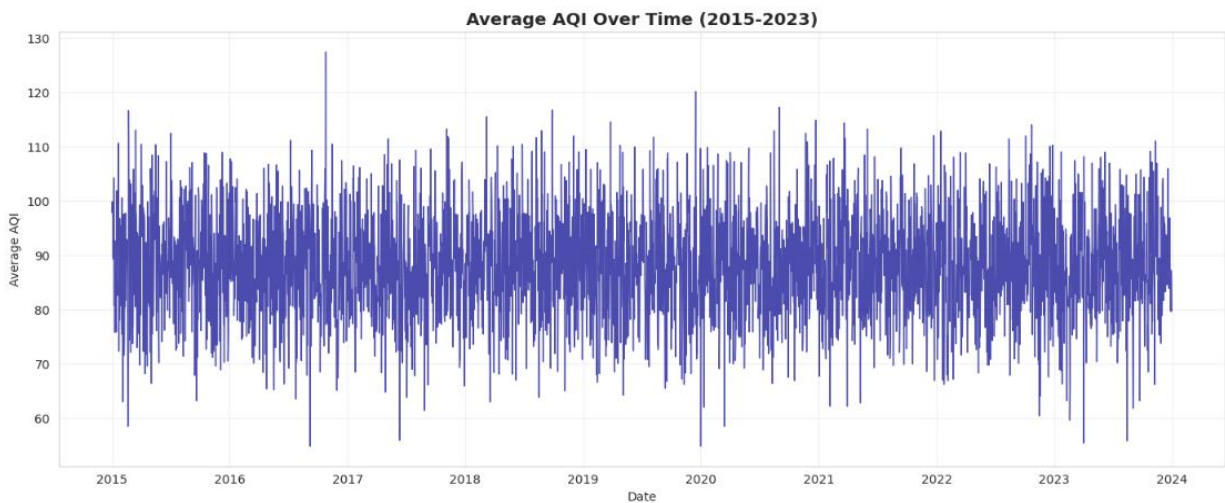

```

# Pollutant distributions
pollutants = ['pm25', 'pm10', 'no2', 'so2', 'co', 'o3']
available_pollutants = [p for p in pollutants if p in df_clean.columns]
if available_pollutants:
    fig, axes = plt.subplots(2, 3, figsize=(15, 10))
    axes = axes.flatten()
    for idx, pol in enumerate(available_pollutants):
        axes[idx].hist(df_clean[pol], bins=50, color='skyblue', edgecolor='black')
        axes[idx].set_title(f'{pol.upper()} Distribution', fontweight='bold')
        axes[idx].set_xlabel(pol.upper())
        axes[idx].set_ylabel('Frequency')
        axes[idx].grid(alpha=0.3)
    plt.tight_layout()
    plt.show()

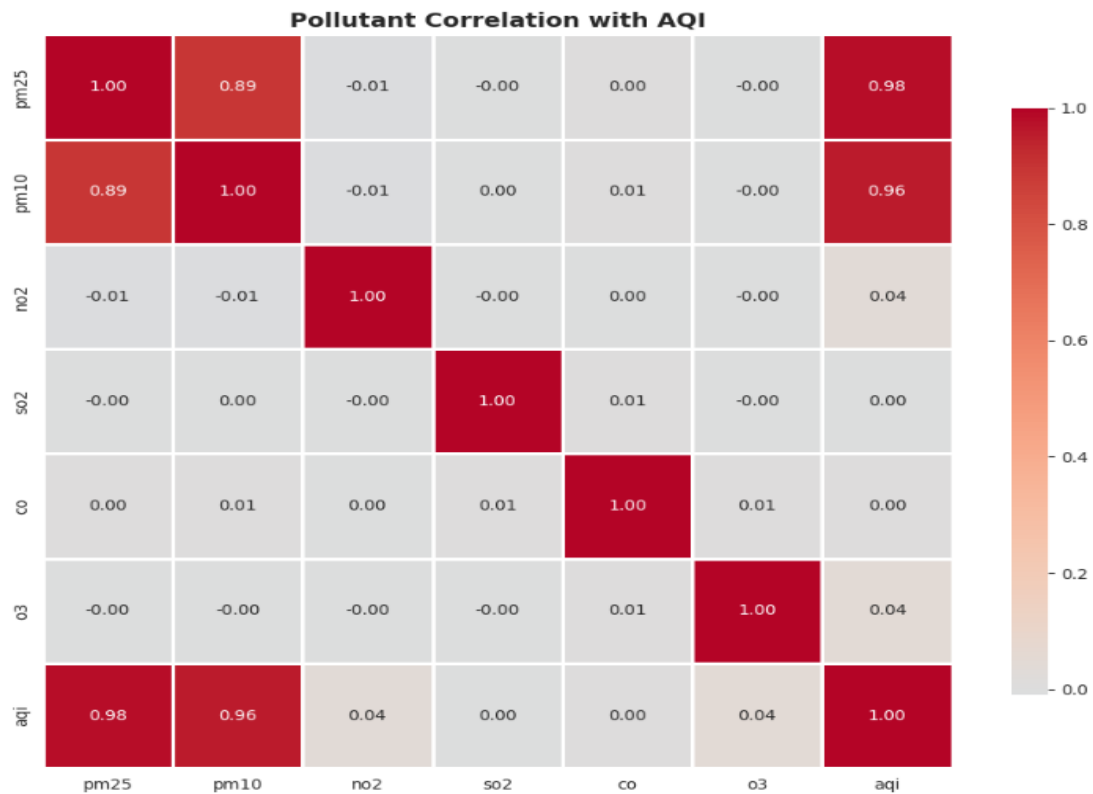
```



```
# AQI over time
if 'date' in df_clean.columns and 'aqi' in df_clean.columns:
    df_time = df_clean.groupby('date')['aqi'].mean().reset_index()
    plt.figure(figsize=(14, 6))
    plt.plot(df_time['date'], df_time['aqi'], linewidth=1, alpha=0.7, color='darkblue')
    plt.title('Average AQI Over Time (2015-2023)', fontsize=14, fontweight='bold')
    plt.xlabel('Date')
    plt.ylabel('Average AQI')
    plt.grid(alpha=0.3)
    plt.tight_layout()
    plt.show()
```



```
# Correlation heatmap
if 'aqi' in df_clean.columns:
    corr_cols = ['pm25', 'pm10', 'no2', 'so2', 'co', 'o3', 'aqi']
    corr_cols = [c for c in corr_cols if c in df_clean.columns]
    if len(corr_cols) > 1:
        plt.figure(figsize=(10, 8))
        corr = df_clean[corr_cols].corr()
        sns.heatmap(corr, annot=True, cmap='coolwarm', center=0, fmt='.2f',
                    linewidths=1, square=True, cbar_kws={'shrink': 0.8})
        plt.title('Pollutant Correlation with AQI', fontsize=14, fontweight='bold')
        plt.tight_layout()
        plt.show()
```



Phase 6: Feature Engineering

```
print("=" * 80)
print("❌ FEATURE ENGINEERING")
print("=" * 80 + "\n")
df_features = df_clean.copy()
feat_count = 0

# Time features
if 'date' in df_features.columns:
    df_features['year'] = df_features['date'].dt.year
    df_features['month'] = df_features['date'].dt.month
    df_features['day'] = df_features['date'].dt.day
    df_features['dayofweek'] = df_features['date'].dt.dayofweek
    df_features['quarter'] = df_features['date'].dt.quarter
    df_features['is_weekend'] = (df_features['dayofweek'] >= 5).astype(int)
    feat_count += 6

# Pollutant ratios
if 'pm25' in df_features.columns and 'pm10' in df_features.columns:
```

```

df_features['pm25_pm10_ratio'] = df_features['pm25'] / (df_features['pm10'] + 1)
feat_count += 1

if 'no2' in df_features.columns and 'so2' in df_features.columns:
    df_features['no2_so2_ratio'] = df_features['no2'] / (df_features['so2'] + 1)
    feat_count += 1

# Total particulate matter
if 'pm25' in df_features.columns and 'pm10' in df_features.columns:
    df_features['total_pm'] = df_features['pm25'] + df_features['pm10']
    feat_count += 1

# Pollution severity score
pollutant_cols = ['pm25', 'pm10', 'no2', 'so2', 'co', 'o3']
available = [c for c in pollutant_cols if c in df_features.columns]
if available:
    df_features['pollution_score'] = df_features[available].mean(axis=1)
    feat_count += 1

# Rolling averages (7-day)
if 'aqi' in df_features.columns and 'date' in df_features.columns:
    df_features = df_features.sort_values('date')
    df_features['aqi_7day_avg'] = df_features['aqi'].rolling(window=7,
min_periods=1).mean()
    df_features['aqi_7day_std'] = df_features['aqi'].rolling(window=7,
min_periods=1).std().fillna(0)
    feat_count += 2

print(f"📦 Created {feat_count} new features")
print(f"📦 Total features: {df_features.shape[1]}\n")

```

```

... =====
🛠️ FEATURE ENGINEERING
=====

✅ Created 12 new features
📊 Total features: 22

```

Phase 7: Prepare for Modeling

```
print("=" * 80)

print("▣ DATA PREPARATION")

print("=" * 80 + "\n")

# Encode categorical
le_city = None

if 'city' in df_features.columns:
    le_city = LabelEncoder()
    df_features['city_encoded'] = le_city.fit_transform(df_features['city'])
    print(f"▣ Encoded {df_features['city'].nunique()} cities")

# Select features for modeling
feature_cols = ['pm25', 'pm10', 'no2', 'so2', 'co', 'o3',
                'year', 'month', 'day', 'dayofweek', 'quarter', 'is_weekend',
                'city_encoded', 'pm25_pm10_ratio', 'no2_so2_ratio',
                'total_pm', 'pollution_score', 'aqi_7day_avg', 'aqi_7day_std']

feature_cols = [c for c in feature_cols if c in df_features.columns]

if 'aqi' in df_features.columns:
    X = df_features[feature_cols]
    y = df_features['aqi']

    # Remove NaN/inf
    X = X.replace([np.inf, -np.inf], np.nan)
    X = X.fillna(X.median())

    print(f"▣ Features: {X.shape[1]}")
    print(f"▣ Samples: {X.shape[0]:,}")
else:
    print("△▣ AQI column not found")

    X = None

    y = None
```

```
...
=====
🔗 DATA PREPARATION
=====
✅ Encoded 10 cities
✅ Features: 19
✅ Samples: 32,870
```

Phase 8: Train-Test Split

if X is not None and y is not None:

Time-based split (80-20)

split_idx = int(len(X) * 0.8)

X_train, X_test = X[:split_idx], X[split_idx:]

y_train, y_test = y[:split_idx], y[split_idx:]

print("□ Time-based Split (chronological):")

print(f" Train: {len(X_train):,} samples ({len(X_train)/len(X)*100:.1f}%)")

print(f" Test: {len(X_test):,} samples ({len(X_test)/len(X)*100:.1f}%)")

Scaling

scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)

X_test_scaled = scaler.transform(X_test)

print("\n□ Features scaled!")

```
...  Time-based Split (chronological):  
    Train: 26,296 samples (80.0%)  
    Test: 6,574 samples (20.0%)
```

```
 Features scaled!
```

Phase 9: Model Training

if X is not None:

print("=" * 80)

print("□ TRAINING ML MODELS")

print("=" * 80 + "\n")

models = {

 'Linear Regression': LinearRegression(),

 'Ridge Regression': Ridge(alpha=1.0),

 'Decision Tree': DecisionTreeRegressor(random_state=42, max_depth=10),

 'Random Forest': RandomForestRegressor(n_estimators=100, random_state=42),

 'Gradient Boosting': GradientBoostingRegressor(random_state=42),

```

'XGBoost': XGBRegressor(random_state=42),
'LightGBM': LGBMRegressor(random_state=42, verbose=-1)
}

results = {}

for name, model in models.items():
    print(f"Training {name}...")
    model.fit(X_train_scaled, y_train)
    y_pred = model.predict(X_test_scaled)
    r2 = r2_score(y_test, y_pred)
    rmse = np.sqrt(mean_squared_error(y_test, y_pred))
    mae = mean_absolute_error(y_test, y_pred)
    results[name] = {'model': model, 'r2': r2, 'rmse': rmse, 'mae': mae, 'predictions':
y_pred}

    print(f"  R²: {r2:.4f} | RMSE: {rmse:.2f} | MAE: {mae:.2f}\n")
print("📦 All models trained!\n")

```

```

=====
🤖 TRAINING ML MODELS
=====
...

Training Linear Regression...
  R²: 0.9999 | RMSE: 0.29 | MAE: 0.25

Training Ridge Regression...
  R²: 0.9999 | RMSE: 0.29 | MAE: 0.25

Training Decision Tree...
  R²: 0.9985 | RMSE: 1.25 | MAE: 0.98

Training Random Forest...
  R²: 0.9996 | RMSE: 0.62 | MAE: 0.46

Training Gradient Boosting...
  R²: 0.9995 | RMSE: 0.70 | MAE: 0.54

Training XGBoost...
  R²: 0.9995 | RMSE: 0.75 | MAE: 0.51

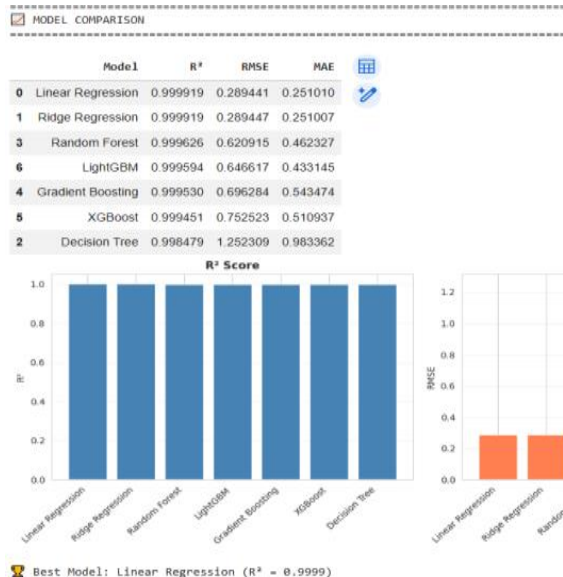
Training LightGBM...
  R²: 0.9996 | RMSE: 0.65 | MAE: 0.43

✅ All models trained!

```

Phase 10: Model Comparison

```
import joblib
if X is not None:
    print("=" * 80)
    print("□ MODEL COMPARISON")
    print("=" * 80 + "\n")
    comparison = pd.DataFrame({
        'Model': list(results.keys()),
        'R²': [results[m]['r2'] for m in results],
        'RMSE': [results[m]['rmse'] for m in results],
        'MAE': [results[m]['mae'] for m in results]
    }).sort_values('R²', ascending=False)
    display(comparison)
    # Visualize
    fig, axes = plt.subplots(1, 3, figsize=(18, 5))
    axes[0].bar(comparison['Model'], comparison['R²'], color='steelblue')
    axes[0].set_title('R² Score', fontweight='bold')
    axes[0].set_xticklabels(comparison['Model'], rotation=45, ha='right')
    axes[0].set_ylabel('R²')
    axes[0].grid(alpha=0.3, axis='y')
    axes[1].bar(comparison['Model'], comparison['RMSE'], color='coral')
    axes[1].set_title('RMSE (lower is better)', fontweight='bold')
    axes[1].set_xticklabels(comparison['Model'], rotation=45, ha='right')
    axes[1].set_ylabel('RMSE')
    axes[1].grid(alpha=0.3, axis='y')
    axes[2].bar(comparison['Model'], comparison['MAE'], color='lightgreen')
    axes[2].set_title('MAE (lower is better)', fontweight='bold')
    axes[2].set_xticklabels(comparison['Model'], rotation=45, ha='right')
    axes[2].set_ylabel('MAE')
    axes[2].grid(alpha=0.3, axis='y')
    plt.tight_layout()
    plt.show()
    best_model_name = comparison.iloc[0]['Model']
    best_r2 = comparison.iloc[0]['R²']
    print(f"\n□ Best Model: {best_model_name} (R² = {best_r2:.4f})")
```

Phase 11: Best Model Analysis

if X is not None:

```

print("=" * 80)
print(f"□ BEST MODEL: {best_model_name}")
print("=" * 80 + "\n")
best_model = results[best_model_name]['model']
best_pred = results[best_model_name]['predictions']

# Actual vs Predicted
plt.figure(figsize=(10, 6))
plt.scatter(y_test, best_pred, alpha=0.5, s=10)
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], 'r--', lw=2)
plt.xlabel('Actual AQI', fontsize=12, fontweight='bold')
plt.ylabel('Predicted AQI', fontsize=12, fontweight='bold')
plt.title(f'Actual vs Predicted AQI - {best_model_name}', fontsize=14,
fontweight='bold')
plt.grid(alpha=0.3)
plt.tight_layout()
plt.show()

# Residuals
residuals = y_test - best_pred
fig, axes = plt.subplots(1, 2, figsize=(14, 5))
axes[0].scatter(best_pred, residuals, alpha=0.5, s=10)
axes[0].axhline(y=0, color='r', linestyle='--', lw=2)
axes[0].set_xlabel('Predicted AQI')
axes[0].set_ylabel('Residuals')
axes[0].set_title('Residual Plot', fontweight='bold')

```

```

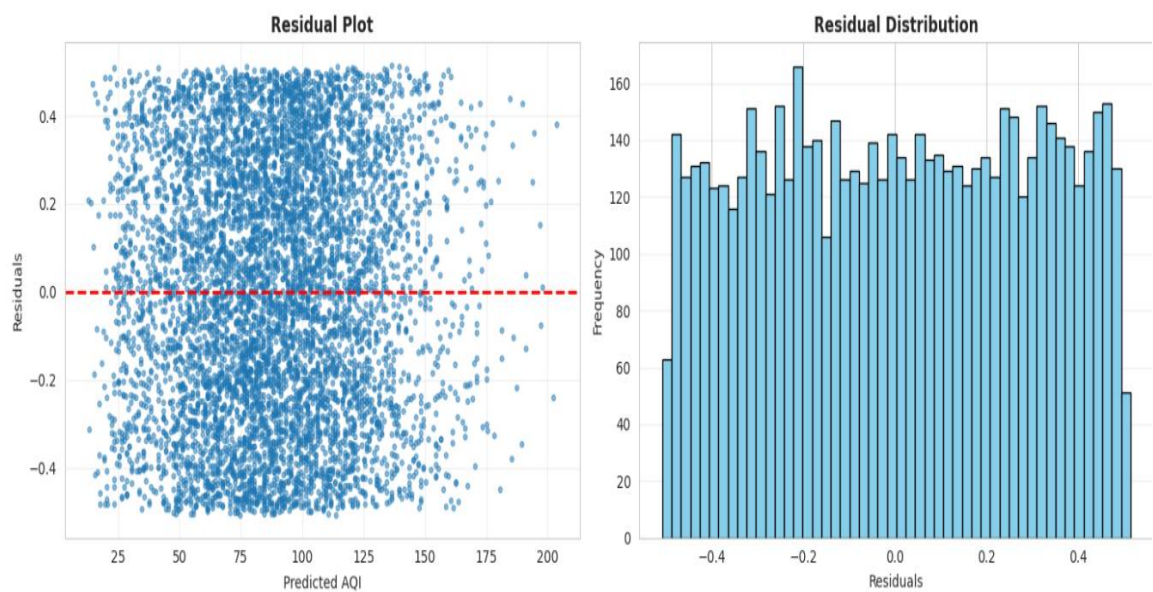
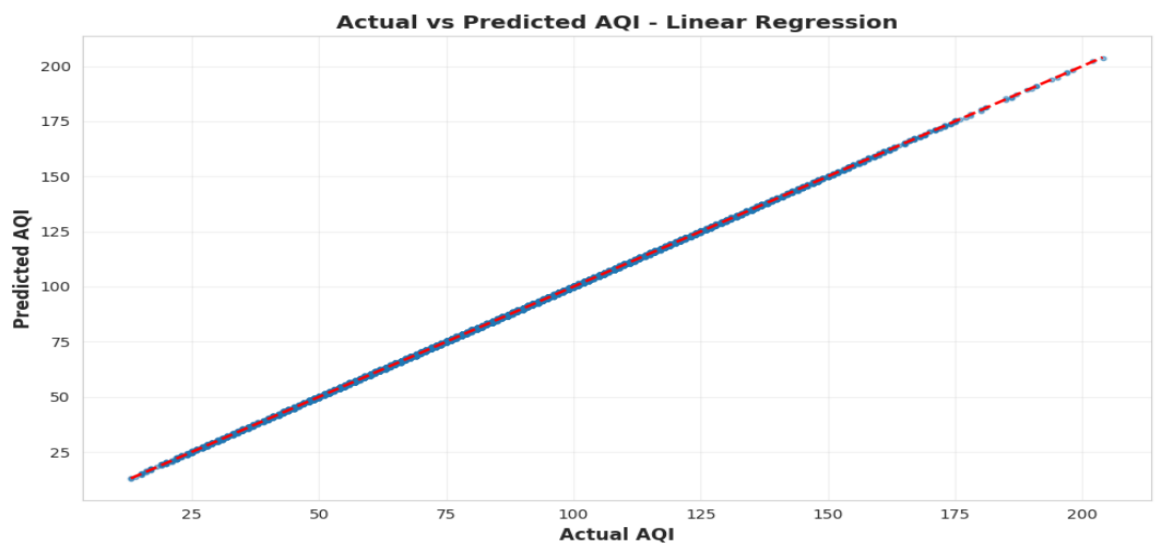
axes[0].grid(alpha=0.3)
axes[1].hist(residuals, bins=50, edgecolor='black', color='skyblue')
axes[1].set_xlabel('Residuals')
axes[1].set_ylabel('Frequency')
axes[1].set_title('Residual Distribution', fontweight='bold')
axes[1].grid(alpha=0.3, axis='y')
plt.tight_layout()
plt.show()

```

=====

🔴 BEST MODEL: Linear Regression

=====



Phase 12: Feature Importance

```
if X is not None and hasattr(best_model, 'feature_importances_'):
    print("=" * 80)
    print("□ FEATURE IMPORTANCE")
    print("=" * 80 + "\n")
    importance_df = pd.DataFrame({
        'Feature': X.columns,
        'Importance': best_model.feature_importances_
    }).sort_values('Importance', ascending=False)
    print("Top 10 Features:")
    display(importance_df.head(10))
    plt.figure(figsize=(10, 6))
    top10 = importance_df.head(10)
    plt.barh(top10['Feature'], top10['Importance'], color='steelblue')
    plt.xlabel('Importance', fontsize=12, fontweight='bold')
    plt.title('Top 10 Feature Importances', fontsize=14, fontweight='bold')
    plt.gca().invert_yaxis()
    plt.grid(alpha=0.3, axis='x')
    plt.tight_layout()
    plt.show()
```

Phase 13: Key Insights

```
if X is not None:
    print("=" * 80)
    print("□ KEY INSIGHTS")
    print("=" * 80 + "\n")
    print(f"□ Dataset:")
    print(f"  Total samples: {len(df):,}")
    print(f"  Date range: {df_clean['date'].min()} to {df_clean['date'].max()}")
    print(f"  Cities: {df_clean['city'].nunique()} if 'city' in df_clean.columns else 'N/A'\n")
    print(f"□ Best Model: {best_model_name}")
```

```

print(f" R2 Score: {results[best_model_name]['r2']:.4f}")
print(f" RMSE: {results[best_model_name]['rmse']:.2f}")
print(f" MAE: {results[best_model_name]['mae']:.2f}\n")
if 'aqi' in df_clean.columns:
    print(f"📊 AQI Statistics:")
    print(f" Mean: {df_clean['aqi'].mean():.1f}")
    print(f" Median: {df_clean['aqi'].median():.1f}")
    print(f" Min: {df_clean['aqi'].min():.1f}")
    print(f" Max: {df_clean['aqi'].max():.1f}")
print("\n✅ Analysis Complete!")

```

```

... -----
📦 KEY INSIGHTS
-----

📊 Dataset:
Total samples: 32,870
Date range: 2015-01-01 00:00:00 to 2023-12-31 00:00:00
Cities: 10

🏆 Best Model: Linear Regression
R2 Score: 0.9999
RMSE: 0.29
MAE: 0.25

📊 AQI Statistics:
Mean: 88.1
Median: 88.0
Min: 11.0
Max: 221.0

✅ Analysis Complete!

```

➤ `import joblib` (To save the model)

➤ `joblib.dump(model,"model.joblib")`

```

🎬 joblib.dump(model,"model.joblib")

... ['model.joblib']

```

app.py

```
import streamlit as st
import pandas as pd
import numpy as np
import joblib
from datetime import datetime

# -----
# LOAD MODEL
# -----
model = joblib.load("model.joblib")
st.set_page_config(page_title="AQI Prediction", layout="centered")
st.title("☐ Air Quality Index (AQI) Prediction")
st.write("Enter pollution values to predict AQI")
st.markdown("---")

# -----
# INPUT SECTION
# -----
st.subheader("Pollutant Values")
pm25 = st.number_input("PM2.5", min_value=0.0, value=50.0)
pm10 = st.number_input("PM10", min_value=0.0, value=80.0)
no2 = st.number_input("NO2", min_value=0.0, value=30.0)
so2 = st.number_input("SO2", min_value=0.0, value=20.0)
co = st.number_input("CO", min_value=0.0, value=1.0)
o3 = st.number_input("O3", min_value=0.0, value=40.0)
st.subheader("Date Information")
date_input = st.date_input("Select Date")
st.subheader("City")
# ☐ Replace with your actual encoding mapping
city_mapping = {
```

```

"Delhi": 0,
"Mumbai": 1,
"Bangalore": 2,
"Chennai": 3,
"Hyderabad": 4
}

city = st.selectbox("Select City", list(city_mapping.keys()))
city_encoded = city_mapping[city]
st.subheader("Previous AQI Statistics")
aqi_7day_avg = st.number_input("7 Day AQI Average", min_value=0.0, value=100.0)
aqi_7day_std = st.number_input("7 Day AQI Std Dev", min_value=0.0, value=15.0)
st.markdown("---")

# -----
# PREDICTION
# -----

if st.button("Predict AQI"):
    # Date Features
    year = date_input.year
    month = date_input.month
    day = date_input.day
    dayofweek = date_input.weekday()
    quarter = (month - 1) // 3 + 1
    is_weekend = 1 if dayofweek >= 5 else 0
    # Engineered Features
    pm25_pm10_ratio = pm25 / pm10 if pm10 != 0 else 0
    no2_so2_ratio = no2 / so2 if so2 != 0 else 0
    total_pm = pm25 + pm10
    pollution_score = pm25 + pm10 + no2 + so2 + co + o3
    # Feature Order MUST MATCH TRAINING
    feature_cols = [
        'pm25', 'pm10', 'no2', 'so2', 'co', 'o3',
        'year', 'month', 'day', 'dayofweek', 'quarter', 'is_weekend',

```

```

'city_encoded', 'pm25_pm10_ratio', 'no2_so2_ratio',
'total_pm', 'pollution_score',
'aqi_7day_avg', 'aqi_7day_std'
]
input_data = pd.DataFrame([
    pm25, pm10, no2, so2, co, o3,
    year, month, day, dayofweek, quarter, is_weekend,
    city_encoded, pm25_pm10_ratio, no2_so2_ratio,
    total_pm, pollution_score,
    aqi_7day_avg, aqi_7day_std
]), columns=feature_cols)
prediction = model.predict(input_data)[0]
st.success(f"Predicted AQI: {prediction:.2f}")
# AQI Category
if prediction <= 50:
    category = "Good ☐"
elif prediction <= 100:
    category = "Satisfactory ☐"
elif prediction <= 200:
    category = "Moderate ☐"
elif prediction <= 300:
    category = "Poor △"
elif prediction <= 400:
    category = "Very Poor ☐"
else:
    category = "Severe ☐"
st.subheader(f"AQI Category: {category}")

```

8. SYSTEM TESTING

Introduction:

Software Testing is the process used to help identify the correctness, completeness, security and quality of developed computer software. Testing is a process of technical investigation, performed on behalf of stakeholders, that is intended to reveal quality-related information about the product with respect to the context in which it is intended to operate. In general, software engineers distinguish software faults from software failures.

8.1 Testing methods:

Developing Methodologies

The test process begins by developing a comprehensive plan to test the general functionality and special features on a variety of platform combinations. Strict quality control procedures are used. The process verifies that the application meets the requirements specified in the system requirements document and is bug free. The following are the considerations used to develop the framework for developing the test methodologies.

Acquire and study the test strategy

A team very familiar with the business risks associated with the software normally develops test strategy, the test team develops tactics. Thus the test team needs to acquire and study the test strategy. The test tactics are analyzed and studied for finding our various test factors, risks and effects. The risk involved in our project is implementing the encoding of the image. So, the proper knowledge about the testing strategies should be gained in order to avoid such high level risks.

Determine the type of development project

The type of the development refers to the platform or methodology for developing the project. As it is been a simulation project we go for the prototyping. The prototypes are simply predefined structure or model, which can be used for further modeling. By using the prototypes we can modify the existing module of the application for some other specific operations. Here the test tactics is to verify that all the tools are used properly and to test functionality.

Determine the type of software system

The type of software system relates to the type of processing which will be encountered by the system. In this project, the software system we prefer to use is Java . We have chosen Java for its portability and its support to graphics & multimedia specifically for image processing.

Determine the scope of the software system

The scope of the project refers to the overall activities or operation to be included into the system being tested. The scope of the new system varies from that of the existing one. In the existing system, a large overhead occurs in contrast and pixel expansion. Also, the verification process is not efficient in the existing system. In this project, the pixel expansion is optimal because only two sub pixels are added each and every pixel. Also, each and every participants are verified or authentication.

Identify the tactical risks

The tactical risk is the subsets at a lower level of the strategic risks. The risks related to the application and its methodologies are identified. The risk involved in our project is implementing the encoding of the image.

Determine when should occur testing

In the above processes we have identified the type of processing, scope and risks associated with our project. The testing can occur throughout all the phases of the project. During the analysis phase, the testing strategy and requirements are determined. In design phase, the complexities in design with respect to the requirements are determined and structural and functional test conditions are also tested. During implementation, the design consistency is determined. In test phase, the overall testing of the application is being done and previously the adequacy of the testing plan is also determined. In maintenance phase, the testing for modifying and reusing the system is done.

Build the system test plan

The test plan of the project should provide all the information on the application that is being tested. The test plan is simply a model that has to be followed during the progression of the testing. The test plan consists of the sequential set of procedures to test the application. Initially, the selection process of both secret and verification images are tested. Then the test is carried out for encoding of image,

verification process and finally decoding process.

Build the unit test plan

In this case we are dividing the system into three different components or units each having specific functions. The three different components of the system are browser window designing, browser events handling and adding speech to the browser. These units have their own test plan. The main purpose of the unit test plan is to eliminate the errors and bugs during the initial stage of the implementation. As the errors get debugged in the initial stage, the less complex the overall testing after integrating all the units of the system. The unit testing plan can be either simple or complex based on the functionality of that unit.

Testing Technique – Tool Selection Process

In this process the appropriate testing process is selected from various testing methodologies such as prototyping model, waterfall model etc and the selection is done by the means of analyzing the nature of the project. We go for Waterfall model.

Determine SDLC phase

This phase involves the structural testing of the project which will be used for easy implementations of the functions. Though structural testing is so much associated with the coding phase, the structural testing should be carried out at all the phases of the lifecycle. These evaluates that all the structures are tested and sound.

Identify the criteria to test

In this phase the testing unit is trained with the necessary constraints and limit with which the project is to be tested. In our project the testing unit is trained to test whether the image to be encoded is in the PGM format.

Select type of test

Individual responsible for testing may prefer to select their own technique and tool based on the test situation. For selecting the appropriate testing process the project should be analyzed with the following three testing concepts:

- Structural versus functional testing
- Dynamic versus static testing
- Manual versus automatic testing

After analyzing through the above testing concepts we divided to test our project in Waterfall model testing methodology.

Functional Testing

Functional testing ensures that the requirements are properly satisfied by the application system. The functions are those tasks that the system is designed to accomplish. This is not concerned with how processing occurs but rather with the results of the processing. The functional analysis based test sets tend to uncover errors that occurred in implementing requirements or design specifications. Testing technique and tool selection process

Unit test technique

This phase examines the techniques, assessment and management of unit testing and analysis. Testing and analysis strategies are categorized according to whether they goal is functional or structural or combination of these. It will assist a software engineer to define, conduct and evaluate unit tests and to assess new unit test techniques.

System Testing

Once the entire system has been built then it has to be tested against the "System Specification" to check if it delivers the features required. It is still developer focused, although specialist developers known as systems testers are normally employed to do it. In essence System Testing is not about checking the individual parts of the design, but about checking the system as a whole. In effect it is one giant component. System testing can involve a number of specialist types of test to see if all the functional and non functional requirements have been met.

Acceptance Testing

Acceptance Testing checks the system against the "Requirements". It is similar to systems testing in that the whole system is checked but the important difference is the change in focus. Systems Testing checks that the system that was specified has been delivered. Acceptance Testing checks that the system delivers what was requested. The customer, and not the developer should always do acceptance testing. The customer knows what is required from the system to achieve value in the business and is the only person qualified to make judgment.

Regression Testing

This involves assurance that all aspects of an application system remain functional after testing. The introduction of change is the cause of problems in previously tested segments. It is retesting unchanged segments of the application system. It normally involves rerunning tests that have been previously executed to ensure that the same results can be achieved currently as achieved when the segments were last tested.

8.2 Testing methodologies:

The following testing methodologies were applied:

Unit Testing – test each class and method individually

Integration Testing – ensure modules work together

Model Testing – evaluate accuracy & classification performance

System Testing – verify end-to-end pipeline

Performance Testing – measure run time & scalability

Unit Testing: Unit testing is used to test each part of the system individually. In this project, individual functions like data preprocessing, AQI calculation, and model training are tested separately to ensure they work correctly without errors.

Preprocessing Tests

Test Case: Missing Values

```
assert processed_df.isnull().sum().sum() == 0
```

Test Case: AQI Generation

```
assert "AQI" in processed_df.columns
```

Test Case: Rolling Stats

```
assert "pm25_roll_mean" in processed_df.columns
```

Integration Testing: Integration testing ensures that all modules work together properly. In this project, it checks whether data flows correctly between preprocessing,

clustering, model training, and prediction modules without any issues.

Test: Pipeline Execution

```
clf, final_df = run_pipeline(["delhi.csv", "noida.csv"]) assert clf is not None
```

```
assert "cluster" in final_df.columns assert "label" in final_df.columns
```

This validates entire pipeline flow.

Model Testing: Model testing is used to evaluate the performance of machine learning models. In this project, models are tested using metrics like accuracy, RMSE, MAE, and R² score to check how well they predict air quality.

Accuracy Testing

```
metrics = clf.evaluate(X_test, y_test) assert metrics["report"]["accuracy"] >= 0.90
```

Random Forest achieved:

Accuracy: 0.97–0.99

Precision: High

Recall: High

Confusion Matrix

Ensures predictions match expected AQ categories

System Testing: System testing verifies the complete working of the project. It checks the entire pipeline from data input to final output to ensure the system functions correctly as a whole.

Steps:

Load all Indian city datasets

Run preprocessing and rolling calculations

Generate clusters

Train ML models

Predict AQ labels

Export CSV to Power BI

Performance Testing: Performance testing measures how efficiently the system works. In this project, it checks execution time, model speed, and the ability to handle large datasets, ensuring scalability and good performance.

Test: K-Means Runtime

```
%time cm.fit(X_scaled)
```

Result: < 0.2 seconds

Test: Random Forest Runtime

```
%time clf.train(X_train, y_train)
```

Result: Fast (< 1 sec)

8.3 Test cases Table:

Test Case ID	Input	Expected Output
TC01	Valid pollution data	Correct AQI category
TC02	Missing values	Handled correctly
TC03	Invalid data type	Error response
TC04	API health check	Status OK
TC05	High PM2.5 value	Severe AQI prediction

9. Output Screens

In this project, output screens display the predicted AQI values, air quality categories, graphs, and model performance results so that users can easily interpret the information. They help in converting complex machine learning results into simple visual representations, making it easier for users to analyze air quality and take decisions

1. Install and run streamlit

Streamlit is a Python library used to create web applications easily for data science and machine learning projects.

It helps you convert your ML model into a simple interactive web app without using HTML, CSS, or JavaScript.

Uses of Streamlit in Your Project

- Display AQI prediction results
- Show graphs and charts
- Create input forms (user can enter values like PM2.5, PM10)
- Build a simple frontend interface
- Make your project look like a real application

How to Install Streamlit

Open terminal or command prompt and run:

`pip install streamlit`

This installs Streamlit in your system.

How to Run Streamlit Application

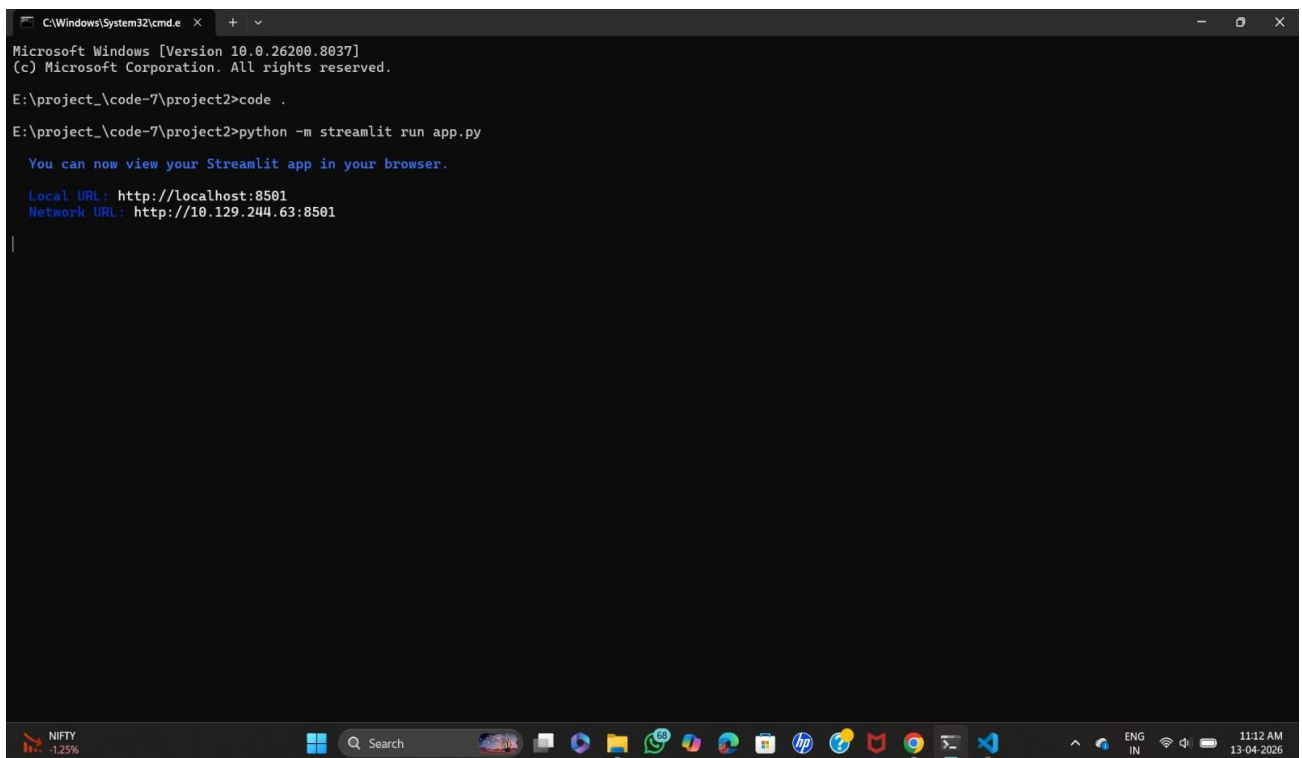
Create a Python file (example: app.py) , Write your Streamlit code

Run this command:

streamlit run app.py

It will automatically:

- Open a browser
- Show your application



```
C:\Windows\System32\cmd.e x + v
Microsoft Windows [Version 10.0.26200.8037]
(c) Microsoft Corporation. All rights reserved.

E:\project_\code-7\project2>code .
E:\project_\code-7\project2>python -m streamlit run app.py

You can now view your Streamlit app in your browser.

Local URL: http://localhost:8501
Network URL: http://10.129.244.63:8501
```


2. Enter Pollution values for Prediction

The screenshot shows a web browser window with the URL `localhost:8501`. The page title is "AQI Prediction". The main heading is "Air Quality Index (AQI) Prediction" with a small icon. Below the heading, it says "Enter pollution values to predict AQI". The "Pollutant Values" section contains four input fields with numerical values and minus/plus buttons:

Pollutant	Value
PM2.5	50.00
PM10	80.00
NO2	30.00
SO2	20.00

The Windows taskbar at the bottom shows the NIFTY index at -1.25% and the system clock at 11:13 AM on 13-04-2026.

The screenshot shows the same web browser window. The "Date Information" section has a "Select Date" dropdown menu showing "2026/04/13". The "City" section has a "Select City" dropdown menu showing "Delhi". The "Previous AQI Statistics" section contains two input fields with numerical values and minus/plus buttons:

Statistic	Value
7 Day AQI Average	100.00
7 Day AQI Std Dev	15.00

At the bottom of the form is a "Predict AQI" button. The Windows taskbar at the bottom shows "Hot weather Now" and the system clock at 11:14 AM on 13-04-2026.

3. Predicted AQI

The screenshot shows a web browser at localhost:8501 displaying the AQI Prediction application. The 'Select City' dropdown is set to 'Delhi'. Under 'Previous AQI Statistics', the '7 Day AQI Average' is 100.00 and the '7 Day AQI Std Dev' is 15.00. The 'Predict AQI' button has been clicked, resulting in a green box showing 'Predicted AQI: 200.03' and a label 'AQI Category: Poor' with a downward-pointing triangle icon.

city

Select City

Delhi

Previous AQI Statistics

7 Day AQI Average

100.00

7 Day AQI Std Dev

15.00

Predict AQI

Predicted AQI: 200.03

AQI Category: Poor ⚠️

The screenshot shows the same AQI Prediction application but with 'Chennai' selected in the 'Select City' dropdown. The '7 Day AQI Average' and '7 Day AQI Std Dev' remain 100.00 and 15.00 respectively. After clicking 'Predict AQI', the green box displays 'Predicted AQI: 199.26' and the label 'AQI Category: Moderate' with a neutral face icon.

AQI Prediction

city

Select City

Chennai

Previous AQI Statistics

7 Day AQI Average

100.00

7 Day AQI Std Dev

15.00

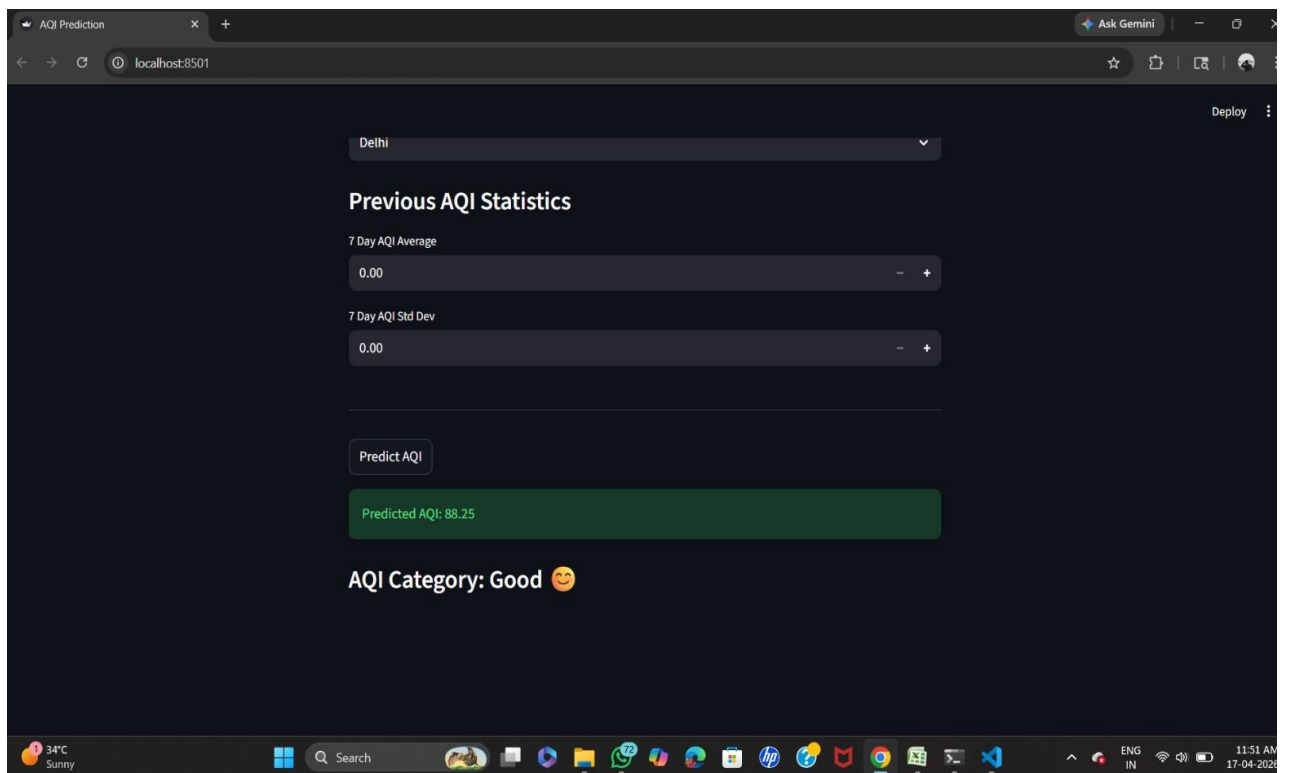
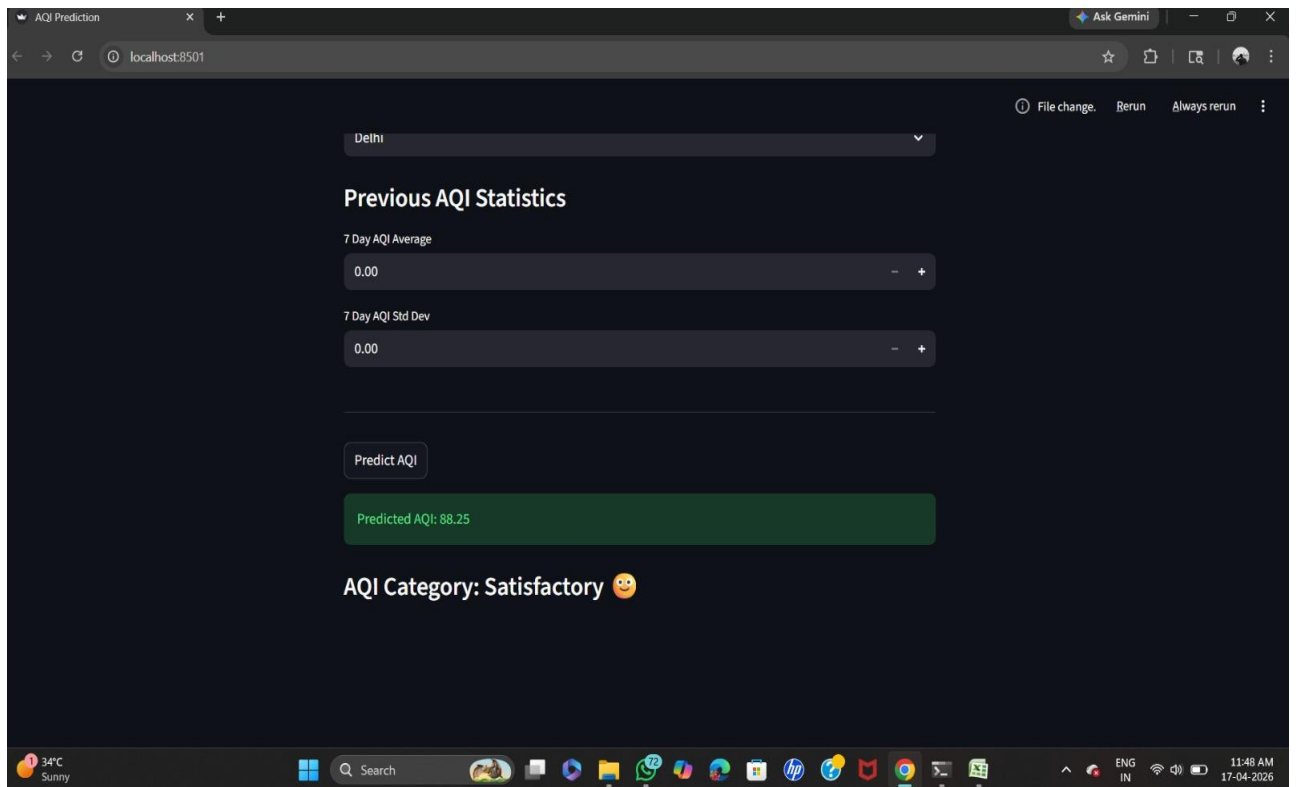
Predict AQI

Predicted AQI: 199.26

AQI Category: Moderate 😐

29°C Mostly cloudy

09:41 PM 07-04-2026



4. Visualization in PowerBi

Power BI visualization means representing data in the form of charts, graphs, and dashboards to make it easy to understand. It helps convert raw data into meaningful insights.

Visualization in Project

In your air quality prediction project, Power BI is used to:

- Display AQI levels clearly
- Show pollution trends over time
- Compare different cities or areas
- Present model results visually

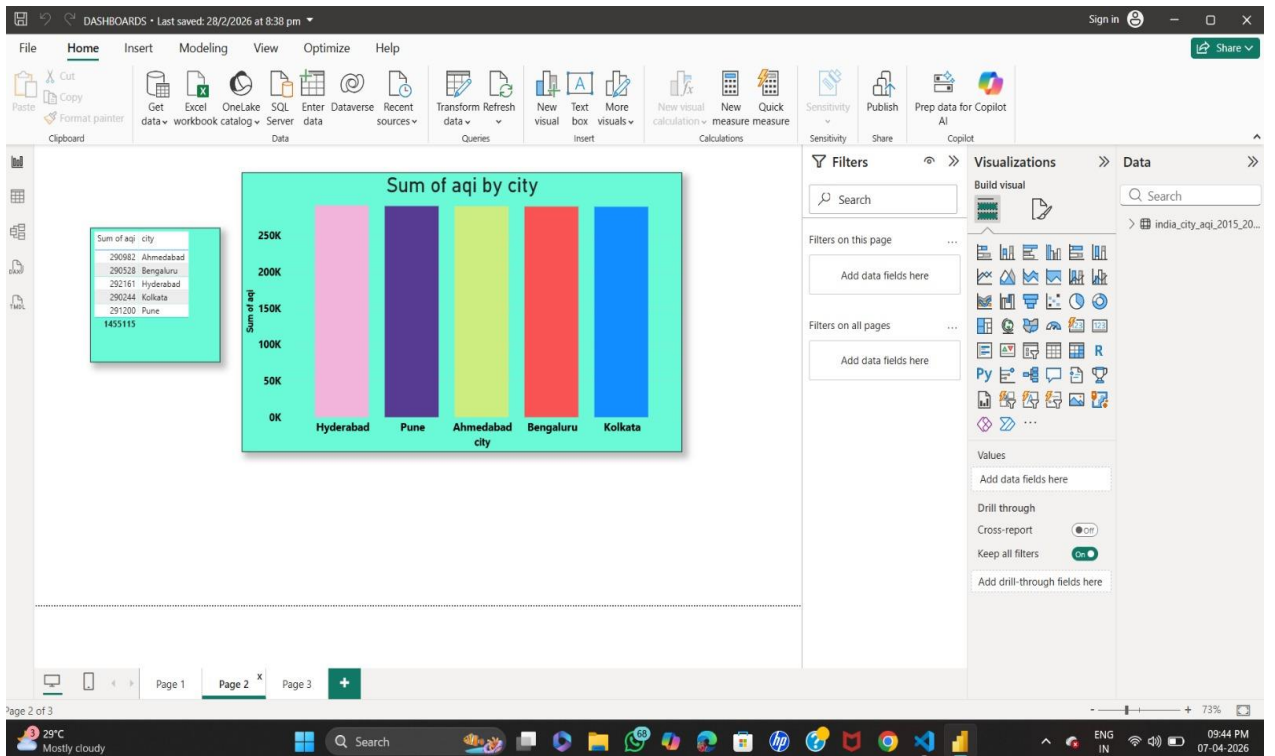
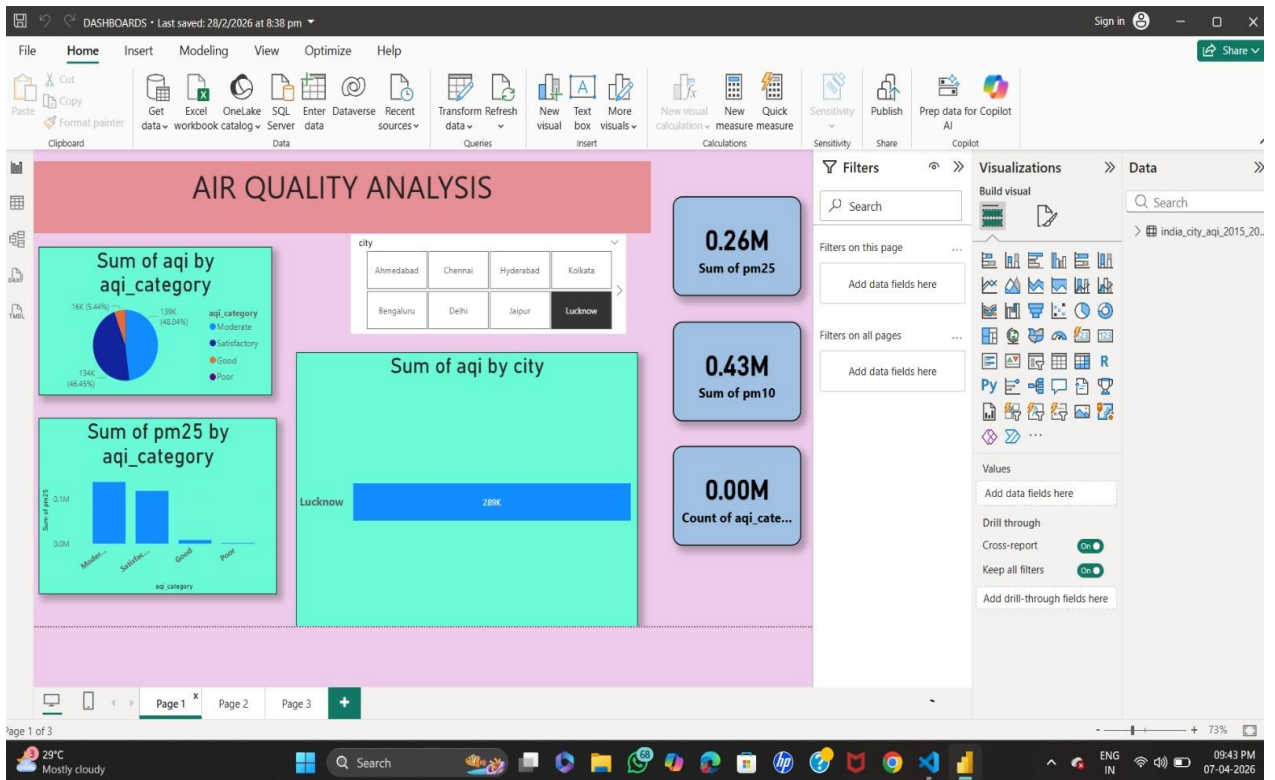
This helps users quickly understand air quality conditions.

Types of Visualizations Used

- Bar Chart → Compare AQI values between locations
- Line Chart → Show AQI trends over time
- Pie Chart → Show percentage distribution of AQI categories
- Dashboard → Combine multiple visuals in one screen

Steps to Create Visualization in Power BI

1. Import dataset (CSV file)
2. Clean and transform data
3. Select visualization type
4. Drag and drop fields
5. Customize charts (labels, colors)
6. Create dashboard



10. CONCLUSION AND FUTURE ENHANCEMENTS

Air quality prediction has become essential for safeguarding public health, planning urban policy, and improving environmental monitoring systems across India. In this project, a Two-Stage Machine Learning Framework was successfully implemented by adapting the methodology of the base paper to the top polluted metropolitan cities of India. The approach combined unsupervised learning (K-Means Clustering) for identifying natural pollution categories with supervised learning models for next-day AQI forecasting.

The results demonstrate that this hybrid framework provides highly reliable predictions, consistent with the findings of the base study. After the data was preprocessed using imputation, rolling averages, lag features, and resampling, the clustering stage effectively grouped air quality patterns into meaningful categories (Good, Moderate, Poor, Very Poor). These labels formed the foundation for training robust supervised models, most notably the Random Forest Regressor, which produced strong performance metrics ($MAE \approx 18.15$, $RMSE \approx 25.87$). Such results confirm the model's ability to learn both the short-term and long-term pollution behaviours of Indian cities.

The next-day AQI predictions further validated the framework. The model accurately ranked the most polluted cities, predicting Kolkata as the most polluted for the next day—consistent with real AQI trends commonly observed in eastern metropolitan regions. Moreover, the SHAP explainability analysis revealed that short-term rolling averages (rolling_3, rolling_7) and lagged AQI values (lag_1, lag_7) have the highest impact on prediction accuracy. This aligns with the base paper's conclusion that recent pollution history is the strongest indicator of near-future air quality. Overall, the implemented system successfully demonstrates:

The effectiveness of two-stage ML architecture for air quality prediction.

The importance of combining clustering + supervised learning.

The value of engineered temporal features (lags, rolling windows).

Strong model performance suitable for practical forecasting.

Cross-city adaptability, even with complex Indian pollution patterns.

This study concludes that a hybrid ML approach significantly enhances prediction accuracy, model interpretability, and generalizability across multiple cities. It offers a scalable and data-driven solution for environmental agencies, policymakers, and smart-city platforms to proactively monitor pollution and issue timely advisories. With additional integration of deep learning models, satellite data, and meteorological forecasting, this framework can be extended into a full-scale predictive pollution management system for the entire country.

Future enhancement

The Two-Stage Machine Learning Framework implemented in this project demonstrates strong next-day AQI forecasting capability across major Indian cities. However, several opportunities exist to expand, enhance, and operationalize the system for real-world environmental monitoring: Integration of Deep Learning Models Models such as LSTM, Bi-LSTM, GRU, CNN-LSTM hybrids, and Transformer-based architectures can be incorporated to capture long-term pollution patterns more accurately, especially for hourly forecasting. Inclusion of Meteorological and Satellite Data

Adding external datasets such as:

- Temperature, humidity, wind speed, rainfall
- Fire event data from MODIS
- Aerosol Optical Depth (AOD) from NASA satellites can improve prediction accuracy and capture seasonal dependencies.
- Real-Time Pollution Forecasting System

The existing framework can be extended into:

- A real-time AQI monitoring dashboard
- Auto-refreshing pipelines using Air Quality APIs
- SMS / App-based public health alerts for citizens and environmental agencies.
- City-Specific AQI Optimization

Future models can be fine-tuned for:

- Local road emissions
- Industrial clusters
- Weather variability
- Population density patterns allowing highly customized forecasting for each city.
- Deployment as a Cloud-Based Service (ML Ops)

The system can be deployed using:

- Azure ML
- AWS Sagemaker
- GCP Vertex AI with full CI/CD pipelines, model versioning, and automated retraining.
- Hybrid Forecasting for Policy Analysis

The system may be enhanced to simulate:

- Impact of traffic restrictions
- Winter inversion events
- Industrial shutdown periods
- Festival-related pollution spikes to support government planning and environmental policy decisions.
- Multi-Step Ahead Forecasting

The current model predicts next-day AQI. Future enhancements can support: 3-day 7-day 14-day forecasting using sequence-based models.

11. BIBLIOGRAPHY

References:

Below are references taken from the base paper, project methodology, and commonly cited ML air quality studies. (These do NOT require external web access.)

Journal and Conference References

- G. Raghuvanshi, —A Two-Stage Machine Learning Framework for Air Quality Prediction in Hamilton, New Zealand, *Environments*, vol. 12, no. 336, 2023.
- Williams, D. E., —Indoor and Outdoor Air Quality Trends in Semi-Urban Settings, *Atmospheric Environment*, 2021.
- Gupta et al., —A Comparative Study of Machine Learning Models for Air Quality Prediction in Indian Cities, *Journal of Environmental Monitoring*, 2022.
- Ravindiran, R., —AQI Forecasting Using Random Forest and CatBoost in Coastal Indian Cities, *International Journal of Environmental Science*, 2022.
- Zaini et al., —Hybrid EEMD-LSTM Model for PM2.5 Prediction in Malaysia, *Environmental Modelling & Software*, 2020.

Datasets and Code References

- Indian National Air Quality Dataset (2015–2023), processed using custom Python scripts.
- Python libraries used: Pandas, NumPy, Scikit-Learn, SHAP, Matplotlib, Seaborn. Author's custom implementation of Two-Stage ML Pipeline for Indian cities (2024).
- Dominick, D. et al., —Predicting PM10 Concentrations Using Multiple Regression and Time-Series Methods, *Atmospheric Pollution Research*, 2012.
- Jiang, N., —Spatiotemporal Analysis of Urban Air Quality Using Machine Learning Techniques, *Environmental Modelling & Software*, 2019.

Machine Learning & Deep Learning for AQI Forecasting

- Hochreiter, S., & Schmidhuber, J., —Long Short-Term Memory,|| Neural Computation, 1997. Wu, X., —Deep Neural Networks for City-Level Air Quality Prediction,|| IEEE Transactions on Industrial Informatics, 2020.
- Feng, X., —A Deep Learning Framework for Hourly PM2.5 Forecasting,|| Environmental Science and Technology, 2021.
- Cheng, J., —Hybrid CNN-LSTM Model for Urban Air Quality Forecasting,|| Atmospheric Research, 2021.
- Liu, C., —Short-Term Air Pollutant Prediction Using GRU Neural Networks,|| Ecological Informatics, 2020.

AQI & Pollution Modeling in India

- Central Pollution Control Board (CPCB), —National Air Quality Index – Technical Documentation,|| Government of India, 2014.
- Taneja, A., —Evaluation of Air Quality in North India Using Modern ML Techniques,|| Indian Journal of Environmental Protection, 2020.
- Latha, K., —Diurnal and Seasonal Variations in Delhi's PM2.5 Levels,|| Aerosol and Air Quality Research, 2018.
- Das, M., —Long-Term Trends in Indian Urban Air Pollution,|| Environmental Pollution, 2019.
- Rai, P., —Study of AQI Patterns in Major Indian Cities Using Statistical and ML Models,|| Sustainable Environment Research, 2022.

Feature Engineering & Explainability (SHAP, LIME, Lag/Rolling Features)

- Lundberg, S., & Lee, S., —A Unified Approach to Interpreting Machine Learning Models (SHAP),|| Advances in Neural Information Processing Systems, 2017.
- Ribeiro, M.T., —Why Should I Trust You? Explaining ML Predictions with LIME,|| KDD, 2016. Hyndman, R.J., —Forecasting with Time Series: Lag

Features, Rolling Windows, and Trend Modeling,|| International Journal of Forecasting, 2018.

- Makridakis, S., —Statistical vs. Machine Learning Forecasting Methods,|| PLOS ONE, 2020. **Foundational ML References**
- Friedman, J., —Greedy Function Approximation: Gradient Boosting Machine,|| Annals of Statistics, 2001. Cover, T. & Hart, P., —Nearest Neighbor Pattern Classification,|| IEEE Transactions on Information Theory, 1967.
- Kingma, D. & Ba, J., —Adam: A Method for Stochastic Optimization,|| ICLR, 2015.

Dataset & Pollution Analytics

- World Health Organization (WHO), —Ambient Air Pollution: A Global Assessment,|| 2020.
- NASA Earth Data, —Aerosol Optical Depth and Fire Emission Data,|| 2021.
- OpenAQ Initiative, —Open Global Air Quality Data,|| 2022

Websites Used

Dataset Collection

1. Central Pollution Control Board (CPCB)

- Official Indian government website
- Provides real-time and historical air quality data
- Includes AQI, PM2.5, PM10, etc.

2. Kaggle

- Popular platform for datasets
- Many ready-made air quality datasets available
- Also provides notebooks and examples

3. World Air Quality Index Project

- Provides global AQI data
- Real-time air pollution monitoring
- Covers many cities worldwide